

Exercise 1

Consider the sets of natural numbers $A = \{1, 2, 3\}$ and $B = \{2, 3\}$. Which of the following statements is true? (Justify your answers!)

- | | |
|---|---|
| 1. $A \subseteq B$; | 7. $\{3\} \in A \cap B$; |
| 2. $B \subseteq A$; | 8. $(1, 2) \in A \times B$; |
| 3. $A \subset \mathbb{N}$; | 9. $(2, 1) \in A \times B$; |
| 4. $a \in A$; | 10. $(1, 2, 3) \in A^3$; |
| 5. $\emptyset \in \mathcal{P}(A)$; | 11. $A \cap B \in \mathcal{P}(A)$; |
| 6. $\{\emptyset\} \in \mathcal{P}(A)$; | 12. $\overline{A} \in \mathcal{P}(A)$. |

Solution of Exercise 1

1. False. Because $1 \notin B$;
2. True. Because $2, 3 \in A$;
3. True. Because $1, 2, 3 \in \mathbb{N}$ (are natural numbers) and $A \neq \mathbb{N}$;
4. False. Because the elements of A are only 1, 2, and 3;
5. True. Because the empty set $\emptyset$ is a subset of any set, and so, it is of A ;
6. False. Because $\{\emptyset\}$ is the singleton set containing $\emptyset$, which is not a subset of A .
7. False. Because $\{3\}$ is the singleton set containing 3, which is neither a member of A nor a member of B ;
8. True. Because $1 \in A$ and $2 \in B$.
9. False. Because $1 \notin B$;
10. True. Because $1, 2, 3 \in A$ and $A^3 = A \times A \times A$;
11. True. Because, as all the elements in $A \cap B$ must also be elements of A , then $A \cap B \subseteq A$;
12. False.

Justification 1: By definition of complement of a set, $\overline{A}$ does not contain elements in A . Thus, $\overline{A}$ is not a subset of A .

Justification 2: $\overline{A} = \{0, 4, 5, 6, \dots\}$ is the complement of A as a subset of $\mathbb{N}$ (which, from the description of the exercise, seems to be the domain of discourse). Thus, $\overline{A}$ is not a subset of A , as $0 \notin A$.

Exercise 2

1. Let $A = \{x, y, z\}$ and $B = \{x, y\}$. Provide the element listing description of the following sets:

- | | |
|--------------------------------|---------------------------------------|
| (a) $A \cup B$ (union) | (c) $\mathcal{P}(B)$ (powerset); |
| (b) $A \cap B$ (intersection); | (d) $A \times B$ (Cartesian product). |

2. Let A, B, C be sets. Provide a formal (“rule of membership”) description of the following sets (for example, a formal description for the Cartesian product is $A \times B = \{(a, b) \mid a \in A \wedge b \in B\}$):

- | | | |
|------------------|--|-------------------------------------|
| (a) $A \cup B$; | (c) $\mathcal{P}(A)$; | (e) $(A \cap C) \times B$; |
| (b) $A \cap B$; | (d) $(A \times B) \cap (C \times B)$; | (f) $\mathcal{P}(\mathcal{P}(A))$. |

Solution of Exercise 2

1. The element description of the above sets is given below

- | | |
|--------------------------------|---|
| (a) $A \cup B = \{x, y, z\}$; | (c) $\mathcal{P}(B) = \{\emptyset, \{x\}, \{y\}, B\}$; |
| (b) $A \cap B = \{x, y\}$; | (d) $A \times B = \{(x, x), (x, y), (y, x), (y, y), (z, x), (z, y)\}$. |

2. Note that also alternative formulations might be correct.

- | |
|--|
| (a) $A \cup B = \{x \mid x \in A \vee x \in B\}$; |
| (b) $A \cap B = \{x \mid x \in A \wedge x \in B\}$; |
| (c) $\mathcal{P}(A) = \{A' \mid A' \subseteq A\}$; |
| (d) $(A \times B) \cap (C \times B) = \{(x, y) \mid x \in A \wedge x \in C \wedge y \in B\}$; |
| (e) $(A \cap C) \times B = \{(x, y) \mid x \in A \wedge x \in C \wedge y \in B\}$; |
| (f) $\mathcal{P}(\mathcal{P}(A)) = \{F \mid F \subseteq \mathcal{P}(A)\}$ |

Exercise 3

Write the following informal statements by using the formal language of first order logic.

1. There exists an integer m such that $m + m$ is greater than 8;
2. Every natural number can be either even or odd;
3. For all $\varepsilon > 0$, there exists a natural number N such that, for all $n \geq N$, $|x - x_n| \leq \varepsilon$.

Solution of Exercise 3

Note that also alternative slightly different formulations might be correct.

1. $\exists m \in \mathbb{Z}. m + m > 8$;
2. $\forall n \in \mathbb{N}. (\exists m \in \mathbb{N}. (n = 2m) \vee (n = 2m + 1))$;
3. $\forall \varepsilon > 0. (\exists N \in \mathbb{N}. (\forall n \geq N. |x - x_n| \leq \varepsilon))$.

Exercise 4

Provide examples of relations that satisfy the following conditions:

1. Reflexive and symmetric but not transitive;
2. Reflexive and transitive but not symmetric;
3. Symmetric and transitive but not reflexive.

Solution of Exercise 4

Let $A = \{x, y, z\}$. The following are binary relations on X :

1. $R_1 = \{(x, x), (y, y), (z, z), (x, y), (y, x), (x, z), (z, x)\}$. It is clearly reflexive and symmetric. It is not transitive, because $(y, x), (x, z) \in R_1$ but $(y, z) \notin R_1$;
2. $R_2 = \{(x, x), (y, y), (z, z), (x, y), (y, z), (x, z)\}$. It is clearly reflexive and transitive. It is not symmetric, because $(x, y) \in R_2$ but $(y, x) \notin R_2$;
3. $R_3 = \{(x, x), (y, y), (x, y), (y, x)\}$. It is clearly symmetric and transitive. It is not reflexive, because $(z, z) \notin R_3$.

Exercise 5

Let $C = \{1, 2, 17, 484\}$ and $D = \{x, y, z\}$ and consider the set of pairs

$$R = \{(1, x), (2, y), (17, y), (17, z)\}.$$

1. Does R encode a function from C to D ?
2. Is R a binary relation between C and D ?

3. Is $R \cup \{484\}$ a relation between C and D ?
4. Let $X \setminus Y$ denote the elements of X not present in Y . With this, does $R \setminus \{(17, z)\}$ encode a function from C to D ? If so, is the function total or partial?

Solution of Exercise 5

1. No, because 17 is mapped to both y and z ;
2. Yes, because $R \subseteq C \times D$;
3. No, because $484 \in R \cup \{484\}$ but $484 \notin C \times D$;
4. Yes, it encodes a function. It is partial because 484 is not mapped to any value in D .

Exercise 6*

Prove that, for any two integers $m, n \in \mathbb{Z}$, if $m + n \geq 19$, then $m \geq 10$ or $n \geq 10$.
(Hint: is it easier to provide a constructive argument or use a proof by contradiction?)

Solution of Exercise 6

We prove the statement using a proof by contradiction. Assume that $m + n \geq 19$, and for the sake of finding a contradiction, that both $m, n < 10$. By the latter assumption, $m + n < 20$. Therefore, it must be that $m + n = 19$. By subtracting 10 from both sides of the equation we get $m + n - 10 = 9$; and since $n - 10$ is a negative quantity (because $n < 10$), we also obtain that $m > 9$. This is a contradiction, as there is no integer number that is both $m > 9$ and $m < 10$.

Exercise 7

Find the error in the following proof that all horses are the same color.

CLAIM: In any set of h horses, all horses are the same color.

PROOF: By induction on h .

Base Case: For $h = 1$. In any set containing just one horse, all horses clearly are the same color.

Inductive Step: For $k \geq 1$, assume that the claim is true for $h = k$ and prove that it is true for $h = k + 1$. Take any set H of $k + 1$ horses. We show that all the horses in this set are the same color. Remove one horse from this set to obtain the set H_1 with just k horses. By the induction hypothesis, all the horses in H_1 are the same color. Now replace the removed horse and remove a different one to obtain the set H_2 . By the same argument, all the horses in H_2 are the same color. Therefore, all the horses in H must be the same color, and the proof is complete.

Solution of Exercise 7

The problem is in the inductive step. Consider the case $h = 2$ and set of horses given by $H = \{\text{black}, \text{white}\}$. Although it is true that $H_1 = \{\text{black}\}$ and $H_2 = \{\text{white}\}$ contain only horses of the same color, the conclusion of H having all horses of the same color fails.

Implicitly, the argument used in the proof assumes that there exists at least another horse that can be used to make a comparison of color between the element missing in H_1 and H_2 . However, this assumption fails in the example above.

Exercise 8*

Prove, by induction on $n \geq 1$ that, $4^n + 14$ is divisible by 6.

Solution of Exercise 8

Let $P(n)$ be the property “there exists integer m such that $4^n + 14 = 6m$ ” (this is *the* definition for a number to be divisible by 6).

Base Case: We prove $P(1)$. This follows because $4^1 + 14 = 6 \cdot 3$.

Inductive Step: For $i \geq 1$ assume $P(i)$ is true. Then, as $4^i + 14 = 6m$, by a simple rearrangement we have that $4^i = 6m - 14$. Next we show that $P(i + 1)$ also follows.

$$\begin{aligned} 4^{i+1} + 14 &= 4(4^i) + 14 \\ &= 4(6m - 14) + 14 && \text{(inductive hp.)} \\ &= 24m - 42 \\ &= 6(4m - 7) . \end{aligned}$$

Indeed, from the above, $m' = 4m - 7$ is the integer showing that $P(i + 1)$ holds.

Exercise 9 (optional)

Prove that there are infinitely many prime numbers.

Solution of Exercise 9

Assume, by way of contradiction, that there are a finite number of primes $p_1, p_2, p_3, \dots, p_n$. Let k be the product of all these primes plus one:

$$k = p_1 p_2 p_3 \cdots p_n + 1.$$

The integer k is clearly greater than 1 and because k is an integer, there must exist some prime number P which divides it. However, P cannot be any one of $p_1, p_2, p_3, \dots, p_n$ because if it were then P could divide the difference $k - p_1 p_2 p_3 \cdots p_n$ and $k - p_1 p_2 p_3 \cdots p_n = 1$.

P cannot divide 1 and also be prime (which is by definition greater than one). This is a contradiction. Therefore our assumption is false so there must be an infinite number of prime numbers.

Exercise 1

Let $L_1 = \{aa, bb, bbb\}$ and $L_2 = \{abba, aab, bb\}$ be two languages over the alphabet $\Sigma = \{a, b\}$. What are the strings of the following languages?

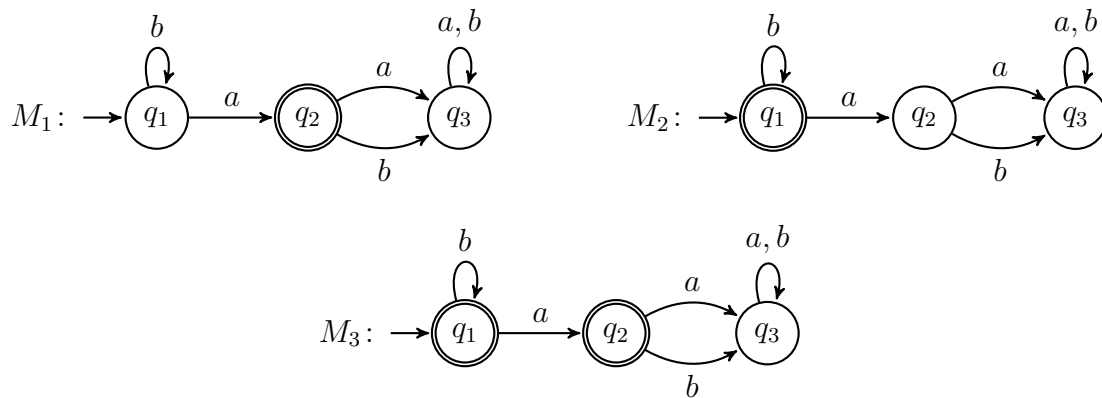
1. $L_1 \circ L_2$
2. $L_1 \cup L_2$
3. $L_1 \cap L_2$
4. Provide a few strings of L_2^*

Solution of Exercise 1

1. $L_1 \circ L_2 = \{aaabba, aaaab, aabb, bbabba, bbaab, bbbb, bbbabba, bbbaab, bbbbbb\}$
2. $L_1 \cup L_2 = \{aa, bb, bbb, abba, aab, bb\}$
3. $L_1 \cap L_2 = \{bb\}$
4. Provide a few strings of L_2^* : $\varepsilon, abba, aab, bb, abbaaab, aabbbb, aabaab, bbb, \dots$

Exercise 2

Given the following three automata M_1 , M_2 , and M_3



1. Describe the sequence of states that M_1 goes through while reading the inputs:
 - (a) $abbbab$
 - (b) $ababaab$
 - (c) $aaaaaa$
 - (d) ε
2. Which of the previous sequences are accepting in M_1 , M_2 and M_3 ?
3. Describe the languages recognised by each of the three automata.

Solution of Exercise 2

1. Describe the sequence of states that M_1 goes through while reading the inputs:

$$(a) \text{ } abbbab: q_1 \xrightarrow{a} q_2 \xrightarrow{b} q_3 \xrightarrow{b} q_3 \xrightarrow{b} q_3 \xrightarrow{a} q_3 \xrightarrow{b} q_3$$

$$(b) \text{ } ababaab: q_1 \xrightarrow{a} q_2 \xrightarrow{b} q_3 \xrightarrow{a} q_3 \xrightarrow{b} q_3 \xrightarrow{a} q_3 \xrightarrow{a} q_3 \xrightarrow{b} q_3$$

$$(c) \text{ } aaaaa: q_1 \xrightarrow{a} q_2 \xrightarrow{a} q_3 \xrightarrow{a} q_3 \xrightarrow{a} q_3 \xrightarrow{a} q_3$$

$$(d) \text{ } \varepsilon: q_1$$

2. Which of the previous sequences are accepting in M_1 , M_2 and M_3 ?

(a) M_1 : None is accepting

(b) M_2 : (d) is accepting

(c) M_3 : (d) is accepting

3. Describe the languages recognised by each of the three automata.

$$(a) \text{ } L(M_1) = \{b^n a \mid n \geq 0\}$$

$$(b) \text{ } L(M_2) = \{b^n \mid n \geq 0\}$$

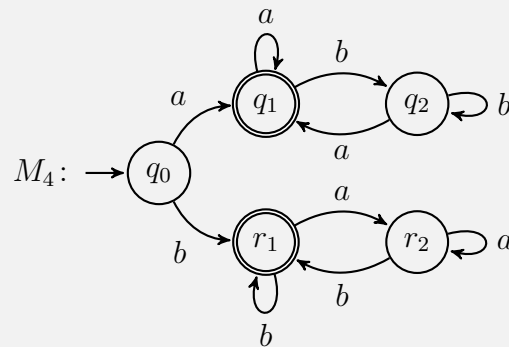
$$(c) \text{ } L(M_3) = L(M_1) \cup L(M_2) = \{b^n a^m \mid n \geq 0, m \in \{0, 1\}\}$$

Exercise 3*

Provide the state diagram of the automaton $M_4 = (Q, \Sigma, \delta, q_0, F)$ given below and describe its language.

	δ	a	b
$Q = \{q_0, q_1, q_2, r_1, r_2\}$	q_0	q_1	r_1
$\Sigma = \{a, b\}$	q_1	q_1	q_2
$F = \{q_1, r_1\}$	q_2	q_1	q_2
	r_1	r_2	r_1
	r_2	r_2	r_1

Solution of Exercise 3



The language recognized by M_4 is $L(M_4) = \{xwx \mid x \in \Sigma, w \in \Sigma^*\} \cup \{a\} \cup \{b\}$. Thus, the automaton M_4 accepts strings that start and end with the same symbol in Σ .

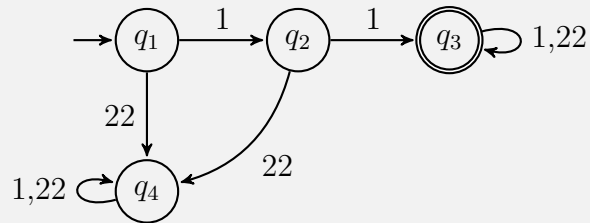
Exercise 4*

For each of the following languages, draw the state diagram of an automaton recognizing it.

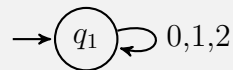
1. $L_1 = \{w \in \{1, 22\}^* \mid w \text{ has } 11 \text{ as prefix}\}$
2. $L_2 = \emptyset \subseteq \{0, 1, 2\}^*$
3. $L_3 = \{\varepsilon\} \subseteq \{0, 1, 2\}^*$
4. $L_4 = \{w \in \{\text{go}, \text{stop}\}^* \mid w = \varepsilon \text{ or ends with } \text{stop}\}$
5. $L_5 = \{w \in \{0, 1\}^* \mid w \text{ has } 001 \text{ as prefix or } 11 \text{ as suffix}\}$

Solution of Exercise 4

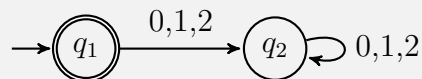
1. $L_1 = \{w \in \{1, 22\}^* \mid w \text{ has } 11 \text{ as prefix}\}$



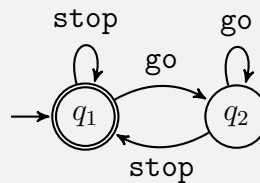
2. $L_2 = \emptyset \subseteq \{0, 1, 2\}^*$



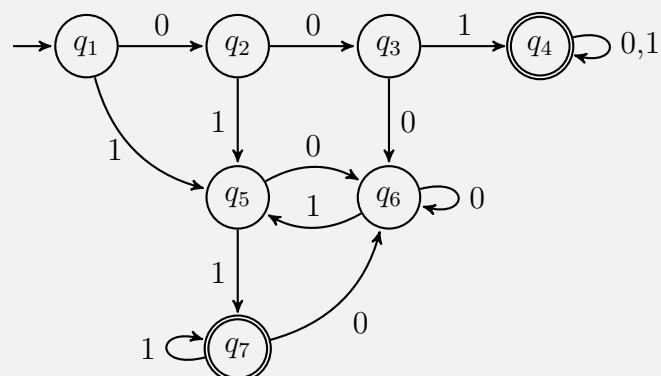
3. $L_3 = \{\varepsilon\} \subseteq \{0, 1, 2\}^*$



4. $L_4 = \{w \in \{\text{go}, \text{stop}\}^* \mid w = \varepsilon \text{ or ends with stop}\}$

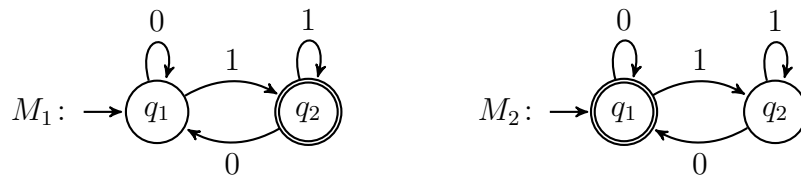


5. $L_5 = \{w \in \{0, 1\}^* \mid w \text{ has } 001 \text{ as prefix or } 11 \text{ as suffix}\}$



Exercise 5*

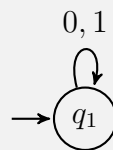
Consider the automata M_1 and M_2 given below.



1. Construct an automaton that recognizes the language $L(M_1) \cap L(M_2)$;
2. Construct an automaton for each of the following languages:
 - (a) $\overline{L(M_1)}$
 - (b) $\overline{L(M_2)}$
 - (c) $\overline{L(M_1) \cap L(M_2)}$

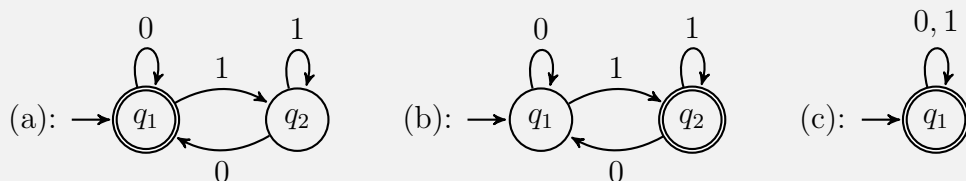
Solution of Exercise 5

1. Notice that $L(M_1) = \{w1 \mid w \in \{0,1\}^*\}$ and $L(M_2) = \{\varepsilon\} \cup \{w0 \mid w \in \{0,1\}^*\}$. Thus, $L(M_1) \cap L(M_2) = \emptyset$ (the language with no strings in it). An automaton recognising $\emptyset$ is the following:



Notice that, there are no accept states. Thus whatever string is given as input, this automaton cannot accept it.

2.



We built the above automata by making all accepting states from the original automaton not accepting, and *vice versa*. In this way, the strings that were accepted in the original automaton will be rejected, and *vice versa*. Start state and transition function are not changed in this process.

Exercise 6

1. Prove that regular languages are closed under intersection.
(Hint: Similar construction to the one for the union, only that $F = F_1 \times F_2$);
2. Prove that regular languages are closed under complement.
(Hint: Transitions don't change. But, what should be now the accepting states?).

Solution of Exercise 6

1. Let $M_1 = (Q_1, \Sigma, q_0^1, \delta_1, F_1)$ and $M_2 = (Q_2, \Sigma, q_0^2, \delta_2, F_2)$ be arbitrary automata. Here we assume without loss of generality that M_1, M_2 are over the same alphabet Σ .

Define the automaton $M = (Q, \Sigma, q_0, \delta, F)$, where

$$\begin{aligned} Q &= Q_1 \times Q_2 & \delta: Q \times \Sigma &\rightarrow Q, \quad \text{where} \\ q_0 &= (q_0^1, q_0^2) & \delta((q_1, q_2), a) &= (\delta_1(q_1, a), \delta_2(q_2, a)) \\ F &= F_1 \times F_2 \end{aligned}$$

Next we prove $L(M) = L(M_1) \cap L(M_2)$.

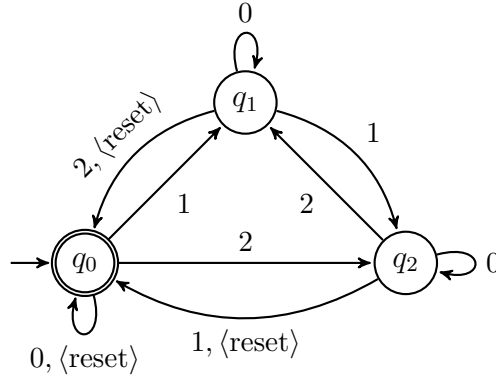
- ($\subseteq$) Let $w \in L(M)$ and $(q_1, r_1)(q_2, r_2) \dots (q_n, r_n)$ be an accepting sequence of states for w in M . By construction $(q_n, r_n) \in F = F_1 \times F_2$, therefore $q_n \in F_1$ and $r_n \in F_2$. Moreover, $q_1 = q_0^1$ and $r_1 = q_0^2$. By definition of δ , for all $j \in \{1, \dots, n-1\}$, it holds that $\delta_1(q_j, a_{j+1}) = q_{j+1}$ and $\delta_2(r_j, a_{j+1}) = r_{j+1}$. By noting that the sequences $q_1, \dots, q_n$ and $r_1, \dots, r_n$ are accepting in M_1 and M_2 , respectively, we obtain that $w \in L(M_1) \cap L(M_2)$.
- ($\supseteq$) Consider $w \in L(M_1) \cap L(M_2)$. Then, $w \in L(M_1)$ and $w \in L(M_2)$ and there exist accepting sequences $q_1, \dots, q_n$ and $r_1, \dots, r_n$ in M_1 and M_2 , respectively. By construction, the sequence $(q_1, r_1)(q_2, r_2) \dots (q_n, r_n)$ is accepting in M . Thus, $w \in L(M)$.
2. Let $M = (Q, \Sigma, \delta, q_0, F)$ be an arbitrary automaton. We want to construct an automaton $\overline{M}$ that recognizes the complement of $L(M)$, where the complement is given by $\overline{L(M)} = \{w \in \Sigma^* \mid w \notin L(M)\}$.

Define the automaton $\overline{M} = (Q, \Sigma, \delta, q_0, Q \setminus F)$. Next we prove that $L(\overline{M}) = \overline{L(M)}$.

- ($\subseteq$) Assume $w \in L(\overline{M})$ and $q_1 q_2 \dots q_n$ be the accepting sequence of states for w in $\overline{M}$. By construction, $q_1 q_2 \dots q_n$ is also the sequence of states for w in M and $q_n \notin F$. Thus, $w \notin L(M)$ which, in turn, is equivalent to $w \in \overline{L(M)}$.
- ($\supseteq$) Consider $w \in \overline{L(M)}$. This is equivalent to say that $w \notin L(M)$. Therefore, the sequence of state $q_1 q_2 \dots q_n$ for w in M is not accepting, meaning that $q_n \notin F$. By construction, $q_1 q_2 \dots q_n$ is an accepting sequence of states for w in $\overline{M}$. Thus, $w \in L(\overline{M})$.

Exercise 7*

Let M be the following automaton over the alphabet $\Sigma = \{0, 1, 2, \langle \text{reset} \rangle\}$.



1. Give examples of accepted and non-accepted words (at least five for each).
2. Define $\text{count}(w)$ to be the sum of the numerical symbols in a string $w \in \Sigma^*$. Prove that $L(M) = \{w \mid (\text{count}(w) \bmod 3) = 0\}$.
(Hint: Note that M keeps a running count of the sum of the numerical input symbols it reads, modulo 3. Every time it receives the $\langle \text{reset} \rangle$ symbol it resets the count to 0).

Solution of Exercise 7

1. **Accepted:** $\varepsilon, 0, \langle \text{reset} \rangle, 2022, 12, 21, \dots$
Non-accepted: $1, 22, 11, 2\langle \text{reset} \rangle 1, 221, \dots$

2. Let $w \in \Sigma^*$. Next we prove, by induction on the length of w , that if the sequence of states in M for the string w ends in q_i , then $\text{counts}(w) = i \pmod{3}$ for $i \in \{0, 1, 2\}$. Note that, since all accepting sequences end in q_0 , by proving this statement we can conclude that $L(M) = \{w \mid \text{count}(w) = 0 \pmod{3}\}$.

Base Case: Assume $|w| = 0$. Then $w = \varepsilon$ is the empty string. The sequence of M for ε is q_0 and $\text{count}(\varepsilon) = 0$. Therefore, $\text{count}(w) = 0 \pmod{3}$.

Inductive Step: For any $|w'| = n \geq 0$, assume $\text{counts}(w') = i \pmod{3}$, whenever the sequence of states in M for w' ends in q_i .

Let $w \in \Sigma^*$ be such that $|w| = n + 1$. Then, $w = w'a$ for some $w' \in \Sigma^*$ and $a \in \Sigma$. Let $s_0, \dots, s_{n-1}$ the sequence of states of M for the string w' . There are three cases, depending whether the ending state s_{n-1} is q_0 , q_1 , or q_2 .

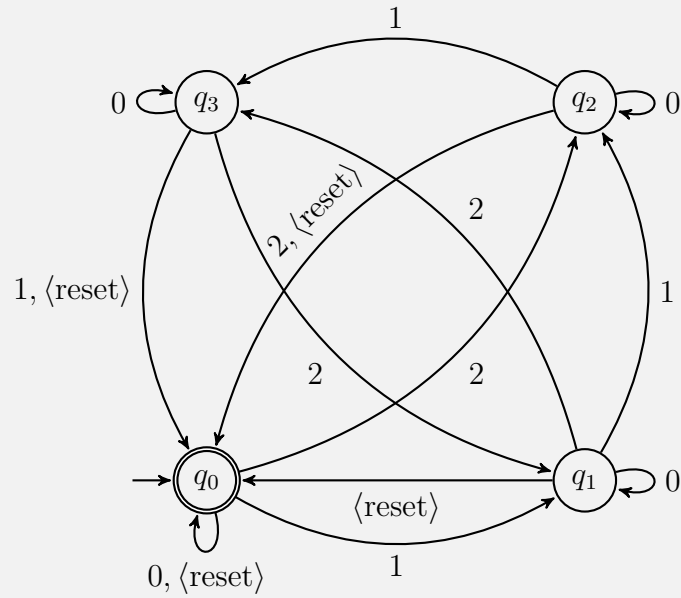
Assume $s_{n-1} = q_0$. By inductive hypothesis, $\text{count}(w') = 0 \pmod{3}$.

- If $a \in \{0, \langle \text{reset} \rangle\}$, the sequence of states in M for w is $s_0, \dots, s_{n-1}, q_0$;
- If $a \in \{1, 2\}$, the sequence of states in M for w is $s_0, \dots, s_{n-1}, q_a$;

In both cases the thesis follows. The cases $s_{n-1} = q_1$ or q_2 are similar.

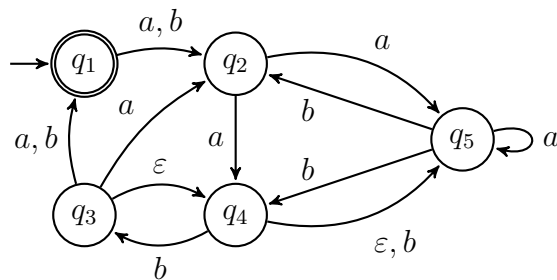
Exercise 8 (optional)

Generalize the automaton M from Exercise 7 so that $L(M) = \{w \mid (\text{count}(w) \bmod 4) = 0\}$.

Solution of Exercise 8

Exercise 1

The following is the state diagram of the nondeterministic automaton N_1 .



1. Provide the formal description of N_1 .
2. Which of the following inputs are accepted? When the input is accepted, justify your answer by providing a corresponding accepting sequence of states in N .

(a) *aabba*

(b) *abbabb*

(c) *aaabab*

(d) *aabbbbba*
3. Describe the computation tree of N_1 for the input string *aabba*.

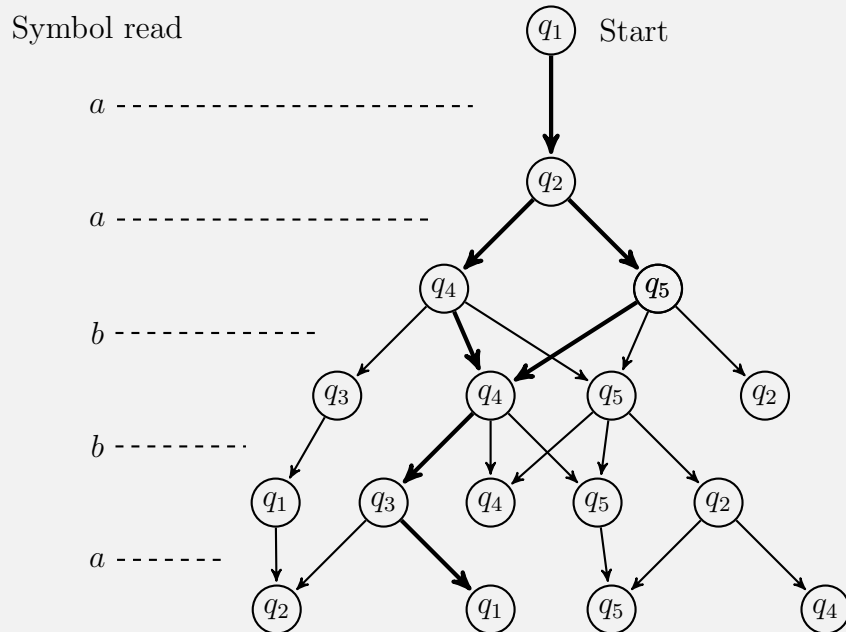
Solution of Exercise 1

1. The automaton N_1 is formally described by the tuple $N_1 = (Q, \Sigma, \delta, q_0, F)$, where:

$Q = \{q_1, q_2, q_3, q_4, q_5\}$	δ	a	b	ε
$\Sigma = \{a, b\}$	q_1	$\{q_2\}$	$\{q_2\}$	$\emptyset$
$\delta: Q \times \Sigma_\varepsilon \rightarrow Q$	q_2	$\{q_4, q_5\}$	$\emptyset$	$\emptyset$
$q_0 = q_1$	q_3	$\{q_1, q_2\}$	$\{q_1\}$	$\{q_4\}$
$F = \{q_1\}$	q_4	$\emptyset$	$\{q_3, q_5\}$	$\{q_5\}$
	q_5	$\{q_5\}$	$\{q_2, q_4\}$	$\emptyset$

2. (a) *aabba* is **accepted**: $q_1 \xrightarrow{a} q_2 \xrightarrow{a} q_4 \xrightarrow{b} q_3 \xrightarrow{\varepsilon} q_4 \xrightarrow{b} q_3 \xrightarrow{a} q_1$;
 (b) *abbabb* is **rejected**;
 (c) *aaabab* is **rejected**;
 (d) *aabbbbba* is **accepted**:
 $q_1 \xrightarrow{a} q_2 \xrightarrow{a} q_4 \xrightarrow{b} q_3 \xrightarrow{\varepsilon} q_4 \xrightarrow{b} q_3 \xrightarrow{\varepsilon} q_4 \xrightarrow{b} q_3 \xrightarrow{\varepsilon} q_4 \xrightarrow{b} q_3 \xrightarrow{a} q_1$;

3. The computation tree of N for the input string $aabba$ is shown below. For convenience, common computation subtree are shared, although, formally, they are separate nondeterministic computations.



Exercise 2*

Provide the state diagram of the nondeterministic automaton $N_2 = (Q, \Sigma, \delta, q_0, F)$, where

$$Q = \{q_0, q_1, q_2, q_3\}$$

$$\Sigma = \{0, 1\}$$

$$F = \{q_1, q_2, q_3\}$$

δ	0	1	ε
q_0	$\emptyset$	$\{q_1, q_2\}$	$\{q_3\}$
q_1	$\{q_3\}$	$\{q_1\}$	$\emptyset$
q_2	$\{q_1, q_2, q_3\}$	$\{q_0, q_1\}$	$\{q_1, q_2\}$
q_3	$\emptyset$	$\{q_2, q_3\}$	$\emptyset$

Which of the following inputs are accepted? When the input is accepted, justify your answer by providing a corresponding accepting sequence of states in N_2 .

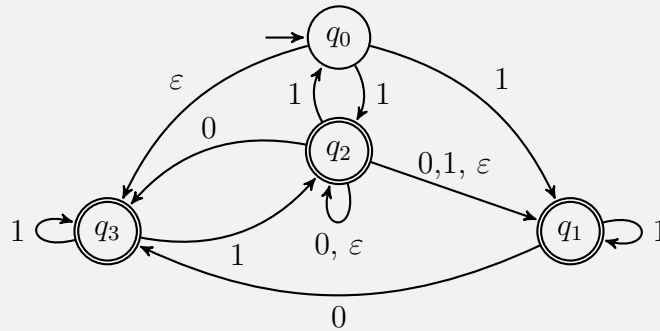
(a) 0011010

(c) 1010111

(b) 0001110

(d) 1101011

Solution of Exercise 2



(a) 0011010 is **rejected**;

(b) 0001110 is **rejected**;

(c) 1010111 is **accepted**: $q_0 \xrightarrow{1} q_1 \xrightarrow{0} q_3 \xrightarrow{1} q_2 \xrightarrow{0} q_3 \xrightarrow{1} q_3 \xrightarrow{1} q_3 \xrightarrow{1} q_3$;

(d) 1101011 is **accepted**: $q_0 \xrightarrow{\varepsilon} q_3 \xrightarrow{1} q_3 \xrightarrow{1} q_2 \xrightarrow{0} q_3 \xrightarrow{1} q_2 \xrightarrow{0} q_3 \xrightarrow{1} q_3 \xrightarrow{1} q_3$;

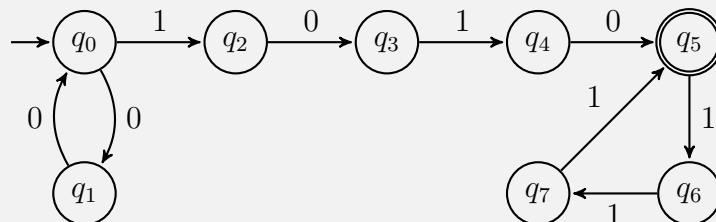
Exercise 3*

For each of the following languages, draw the state diagram of an automaton recognizing it.

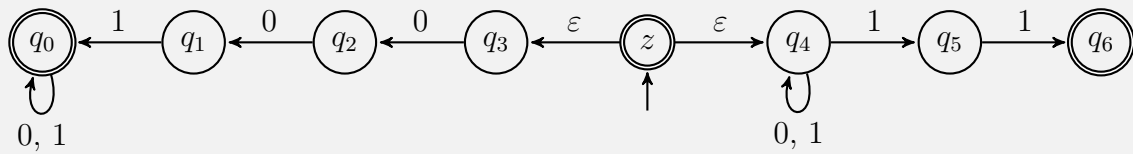
1. $L_1 = \{w \in \{0,1\}^* \mid w = 0^{2n}10101^{3k} \text{ for } n, k \geq 0\}$
2. $L_2 = \{w \in \{0,1\}^* \mid w \text{ has } 001 \text{ as prefix or } 11 \text{ as suffix}\} \cup \{\varepsilon\}$
3. $L_3 = \{w \in \{0,1\}^* \mid w = 0^{2n} \text{ or } w = 1^{3n} \text{ for } n \geq 0 \text{ or } w \text{ has } 101 \text{ as substring}\}$

Solution of Exercise 3

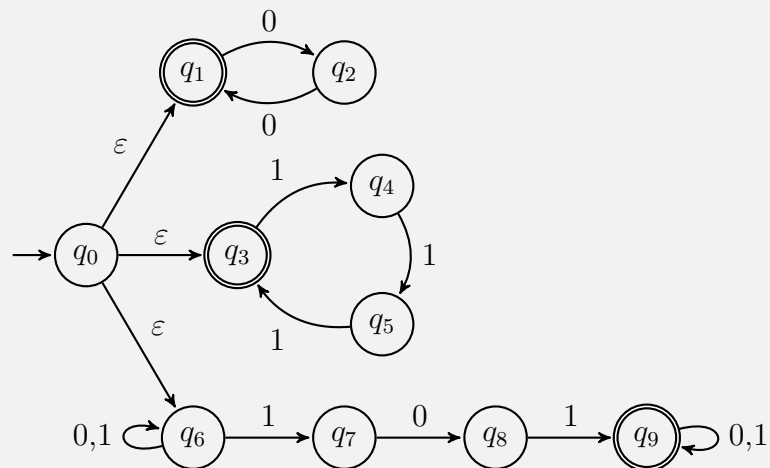
1. $L_1 = \{w \in \{0,1\}^* \mid w = 0^{2n}10101^{3k} \text{ for } n, k \geq 0\}$



2. $L_2 = \{w \in \{0, 1\}^* \mid w \text{ has } 001 \text{ as prefix or } 11 \text{ as suffix}\} \cup \{\varepsilon\}$

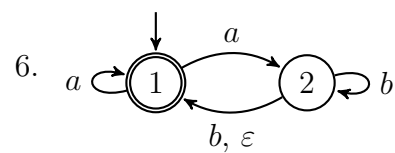
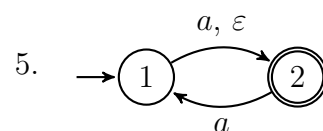
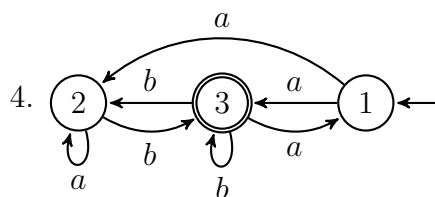
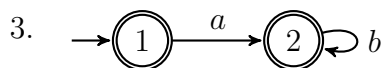
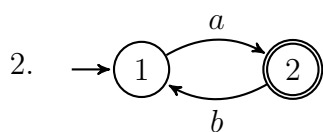
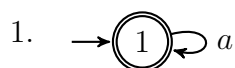


3. $L_3 = \{w \in \{0, 1\}^* \mid w = 0^{2n} \text{ or } w = 1^{3n} \text{ for } n \geq 0 \text{ or } w \text{ has } 101 \text{ as substring}\}$



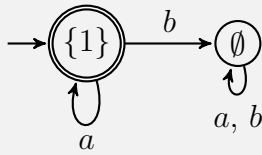
Exercise 4*

Convert each of the following nondeterministic automata over the alphabet $\{a, b\}$ to an equivalent deterministic one using the subset construction. Omit inaccessible states.

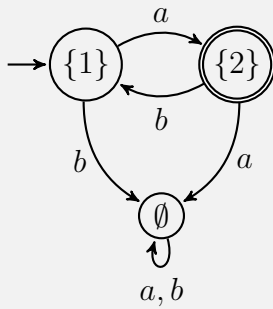


Solution of Exercise 4

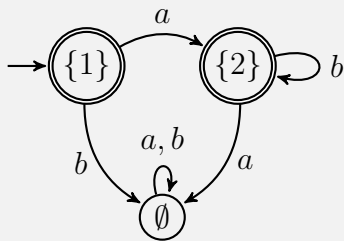
1.



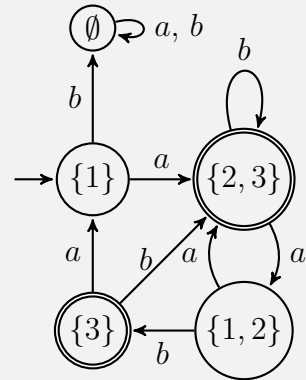
2.



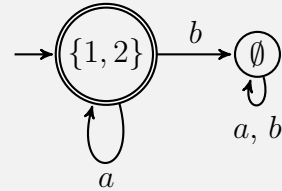
3.



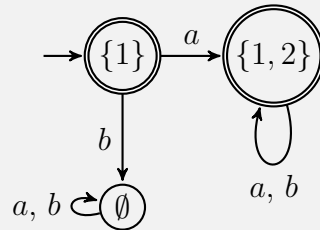
4.



5.

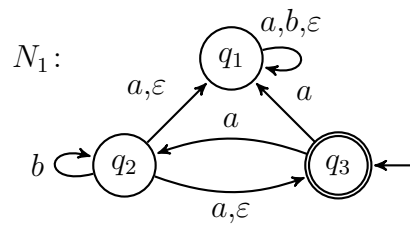


6.

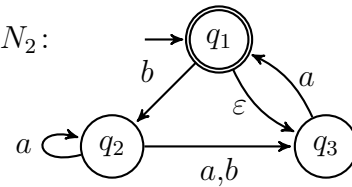


Exercise 5*

Given the following two nondeterministic finite automata N_1 and N_2 ,



N_2 :

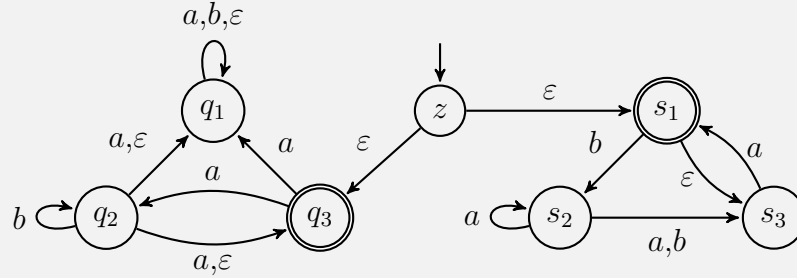


construct an automaton recognising each of the following languages:

1. $L(N_1) \cup L(N_2)$
2. $L(N_1) \circ (L(N_2) \cup L(N_2))$
3. $(L(N_1) \cup L(N_2)) \circ L(N_2)$
4. $(L(N_1) \cup L(N_2)^*) \circ L(N_2)$

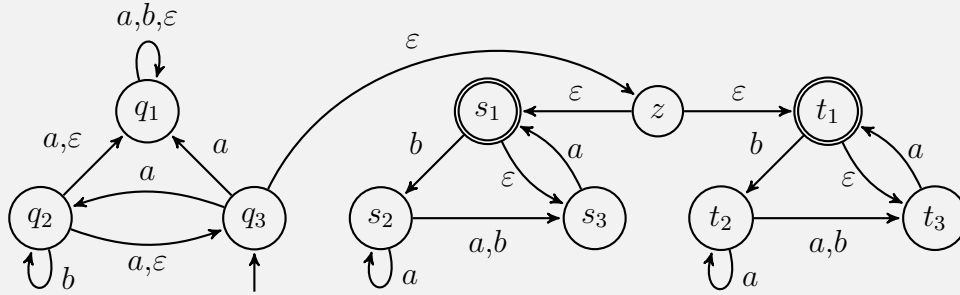
Solution of Exercise 5

1.



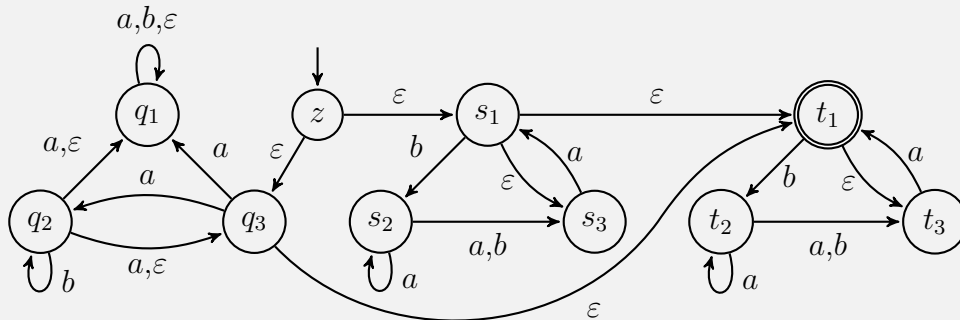
The definition above follows the construction in Theorem 1.45 (M. Sipser, *Introduction to the Theory of Computation*, 3rd ed.).

2.



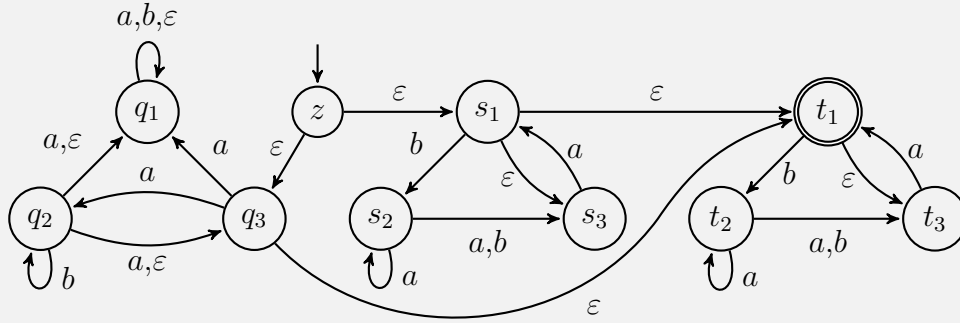
The definition above follows the constructions in Theorems 1.45 and 1.47 (M. Sipser, *Introduction to the Theory of Computation*, 3rd ed.).

3.



The definition above follows the constructions in Theorems 1.45 and 1.47 (M. Sipser, *Introduction to the Theory of Computation*, 3rd ed.).

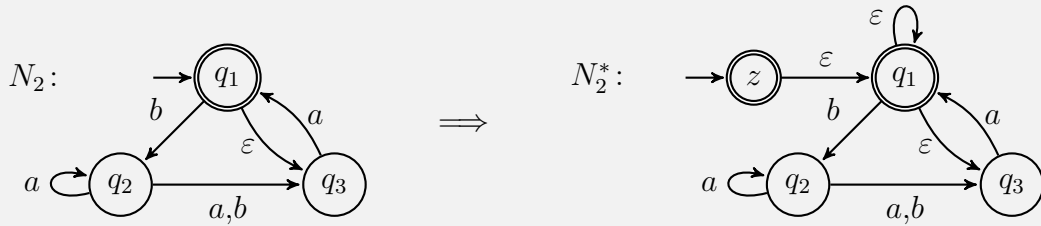
4.



The definition above follows, up to some minor simplifications, the constructions in Theorems 1.45, 1.47, and 1.49 (M. Sipser, *Introduction to the Theory of Computation*, 3rd ed.).

Note that the resulting automaton is the same as the one in the solution of Exercise 5(3). The reason the two solutions are the same is that the language $L(N_2)$ recognised by N_2 is already closed under star operation, i.e., $L(N_2) = L(N_2)^*$. This can be seen from the fact that the only accept state in N_2 is also the starting one.

By applying the construction from Theorem 1.49 (M. Sipser, *Introduction to the Theory of Computation*, 3rd ed.) to the NFA N_2 , we get the following:



By a careful inspection, we notice that two simplifications are applicable in N_2^* :

- the ε -loop in q_1 can be removed, as it does not change the accepting computation sequences passing through that state;
- also the transition $z \xrightarrow{\varepsilon} q_1$ can be removed, as all the strings accepted starting from z are the same as if we were starting from q_1 . After removing the transition, q_1 becomes the start state.

By apply the suggested simplifications we get N_2 back.

To conclude, notice that, observation (a) is always true (ε -loops are useless); observation (b) holds only because the only accept state in N_2 is also the starting state.

Exercise 6 (optional)

The reverse of a string w , denoted $w^{\mathcal{R}}$, is w written backwards; for example, $aab^{\mathcal{R}} = baa$. For a language $A \subseteq \Sigma^*$, define

$$A^{\mathcal{R}} = \{w^{\mathcal{R}} \mid w \in A\}.$$

For example $\{a, ab, aab, aaab\}^{\mathcal{R}} = \{a, ba, baa, baaa\}$.

Show that for any language $A \subseteq \Sigma^*$, if A is regular, then so is $A^{\mathcal{R}}$.

Solution of Exercise 6

Assume $A \subseteq \Sigma^*$ is a regular language. Then, there exists a deterministic finite automaton $M = (Q, \Sigma, \delta, q_0, F)$ such that $A = L(M)$.

For $s \notin Q$, define the *nondeterministic* automaton $N = (Q^{\mathcal{R}}, \Sigma, \delta^{\mathcal{R}}, s, F^{\mathcal{R}})$, such that

$$\begin{aligned} Q^{\mathcal{R}} &= Q \cup \{s\} \\ \delta^{\mathcal{R}}: Q^{\mathcal{R}} \times \Sigma_{\varepsilon} &\rightarrow Q^{\mathcal{R}} \\ F^{\mathcal{R}} &= \{q_0\} \end{aligned} \quad \delta^{\mathcal{R}}(q', x) = \begin{cases} \{q \mid \delta(q, x) = q'\} & \text{if } q' \in Q \text{ and } x \in \Sigma \\ F & \text{if } q' = s \text{ and } x = \varepsilon \\ \emptyset & \text{otherwise} \end{cases}$$

In the definition above, N is “reversing” the transitions in M and switching the role of final/initial states. Next we prove $L(N) = L(M)^{\mathcal{R}}$.

($\supseteq$) Let $w = x_1, \dots, x_n$ be a string over Σ , where each $x_i \in \Sigma$, and assume it is accepted by the computation sequence $p_0, \dots, p_n$ in M . Then, for each $i \in \{0, \dots, n-1\}$, $\delta(p_i, x_{i+1}) = p_{i+1}$. By definition of $\delta^{\mathcal{R}}$ we have $p_i \in \delta^{\mathcal{R}}(p_{i+1}, x_{i+1})$. Moreover, as $p_0 = q_0$ and $p_n \in F$, we have that $p_0 \in F^{\mathcal{R}}$ and $p_n \in \delta^{\mathcal{R}}(s, \varepsilon)$. This means that, $s, p_n, \dots, p_0$ is an accepting sequence for $x_n, \dots, x_1$ in N . As $x_n, \dots, x_1 = w^{\mathcal{R}}$, we just showed that, whenever $w \in L(M)$, then $w^{\mathcal{R}} \in L(N)$. Thus, $L(M)^{\mathcal{R}} \subseteq L(N)$.

($\subseteq$) Let $w = x_1, \dots, x_n$ be a string over Σ , where each $x_i \in \Sigma$. By construction, whenever w is accepted by a sequence $r_0, r_1, \dots, r_n$ in N , we have that $r_0 = s, r_1, \dots, r_n \in Q$ (in particular, $r_1 \in F, r_n = q_0$) and $r_{i+1} \in \delta^{\mathcal{R}}(r_i, x_i)$ for $i \in \{1, \dots, n-1\}$. By definition of $\delta^{\mathcal{R}}$, $r_i = \delta(r_{i+1}, x_i)$, for $i \in \{1, \dots, n-1\}$. Thus $r_n, \dots, r_1$ is a computation in M accepting the string $x_n, \dots, x_1$. As $x_n, \dots, x_1 = w^{\mathcal{R}}$ and $w = (w^{\mathcal{R}})^{\mathcal{R}}$, we have just proved that whenever $w \in L(N)$, then $w \in L(M)$. Thus, $L(N) \subseteq L(M)^{\mathcal{R}}$.

By $L(N) = L(M)^{\mathcal{R}}$ and $A = L(M)$, we obtain that $L(N) = A^{\mathcal{R}}$. By Corollary 1.40 in (M. Sipser, *Introduction to the Theory of Computation*, 3rd ed.), $A^{\mathcal{R}}$ is regular.

Exercise 1

Give regular expressions for each of the following languages over $\{a, b\}$.

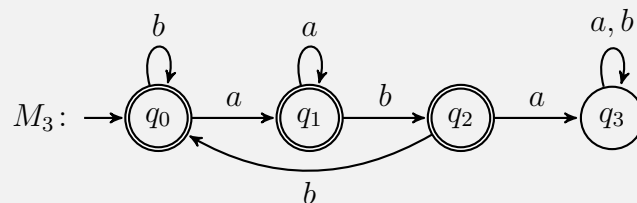
1. $\{w \mid w \text{ contains an even number of } a\text{'s}\}$;
2. $\{w \mid w \text{ contains an odd number of } b\text{'s}\}$;
3. $\{w \mid w \text{ contains an even number of } a\text{'s or an odd number of } b\text{'s}\}$;
4. $\{w \mid w \text{ does not contain the substring } a\}$;
5. $\{w \mid w \text{ does not contain the substring } ab\}$;
6. $\{w \mid w \text{ does not contain the substring } aba\}$.

Try to simplify the expressions as much as possible using the algebraic laws of regular expressions.

Solution of Exercise 1



1. $(b \cup ab^*a)^*$. This expression originated by mimicking the transition structure of the DFA M_1 above, accepting strings with an even number of a 's. (One could also apply the conversion algorithm. See Exercise 3).
2. $(a^*ba^*b)^*a^*ba^*$. This expression originated by mimicking the transition structure of the DFA M_2 above, accepting strings with an odd number of b 's.
3. $(b \cup ab^*a)^* \cup (a^*ba^*b)^*a^*ba^*$. This is the union of the languages above.
4. b^* . This is the language of containing the empty string or strings with only b 's.
5. b^*a^* . This is the language that do not allow ab (in this order).
6. $((aa^*b \cup \varepsilon)b)^*(\varepsilon \cup aa^*(\varepsilon \cup b))$. This expression originated by mimicking the transition structure of the DFA M_3 below, accepting only strings with no occurrences of aba .



Exercise 2*

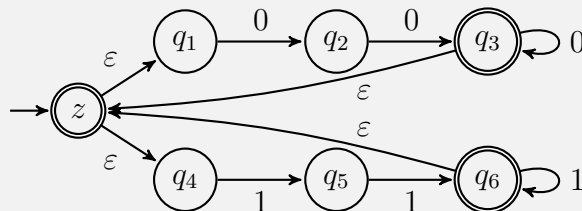
Covert each of the following regular expressions into equivalent NFAs, i.e., recognising the language of strings represented by the regular expression.

1. $(000^* \cup 111^*)^*$;
2. $(01 \cup 10)(01 \cup 10)$;
3. $(0 \cup 1(01^*0)^*1)^*$.

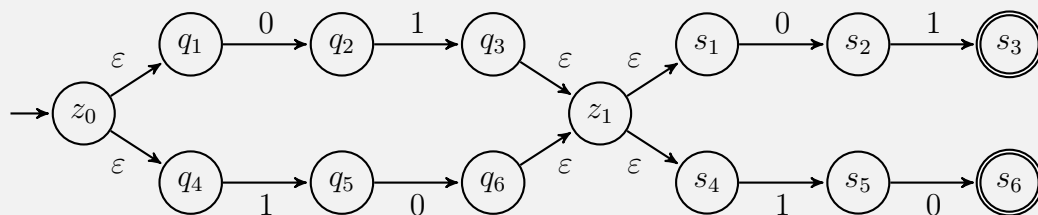
Solution of Exercise 2

Below we apply the standard conversion algorithm but with some extra simplifications.

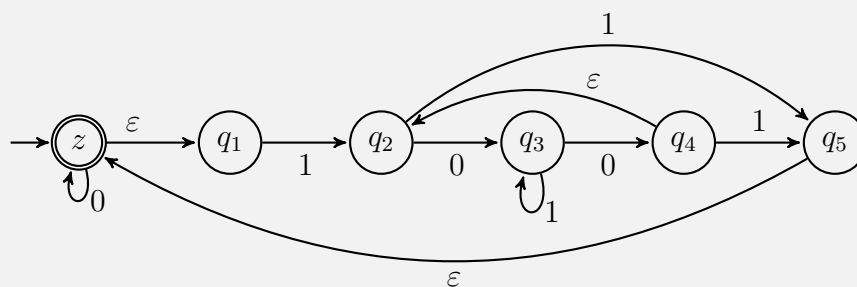
1.



2.

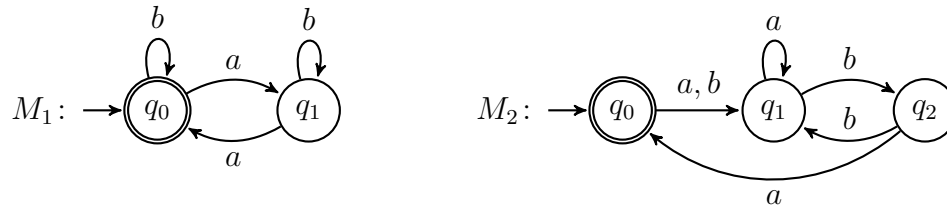


3.



Exercise 3

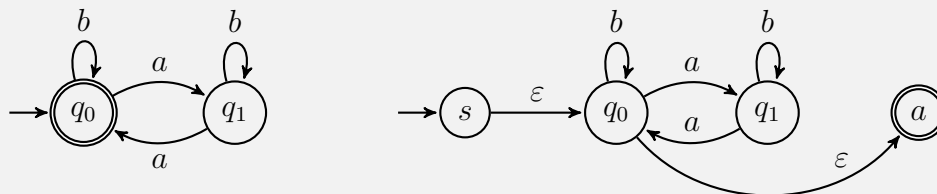
Convert the following DFAs into equivalent regular expressions by using the construction described in Lemma 1.60 (M. Sipser, *Introduction to the Theory of Computation*, 3ed).



Show all the steps of the conversion.

Solution of Exercise 3

1. (Conversion of M_1) In the following $\emptyset$ -transitions on GNFA's are omitted.



1. Starting DFA

2. into GNFA

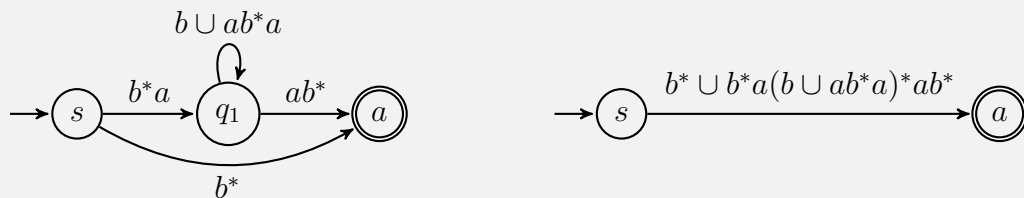


3. removal of q_1

4. removal of q_0

Thus, the regular expression representing $L(M_1)$ is $(b \cup ab^*a)^*$.

Observe that by removing nodes in a different order you might get a different (but equivalent!) regular expression. For example, if in step 3 we remove q_0 we get

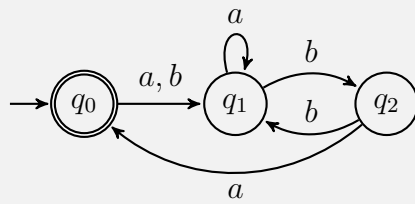


3. removal of q_0

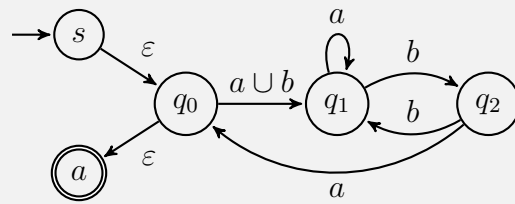
4. removal of q_1

and the final regular expression would be $b^* \cup b^*a(b \cup ab^*a)^*ab^*$.

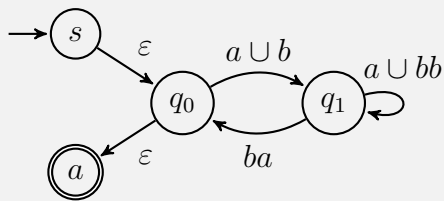
2. (Conversion of M_2) In the following $\emptyset$ -transitions on GNFA's are omitted.



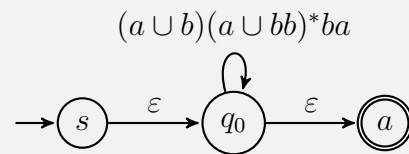
1. Starting DFA



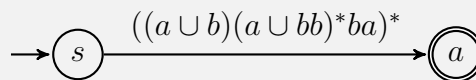
2. into GNFA



3. removal of q_2



4. removal of q_1

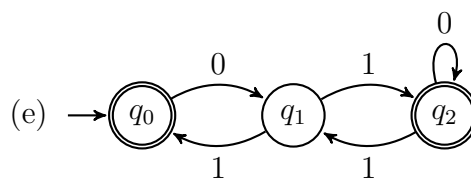
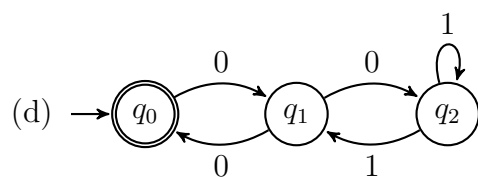
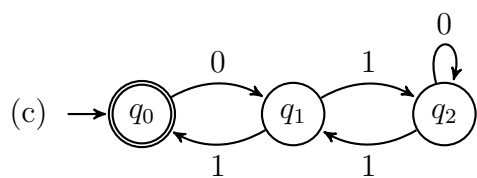
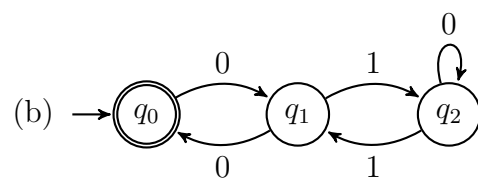
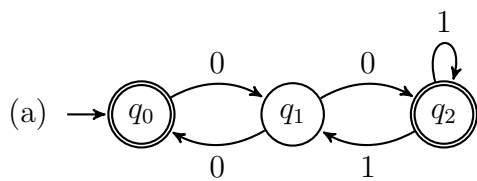


5. removal of q_0

Thus, the regular expression representing $L(M_2)$ is $((a \cup b)(a \cup bb)^*ba)^*$.

Exercise 4*

Match each NFA with an equivalent regular expression.



1. $\varepsilon \cup 0(01^*1 \cup 00)^*01^*$
2. $\varepsilon \cup 0(10^*1 \cup 10)^*10^*$
3. $\varepsilon \cup 0(10^*1 \cup 00)^*0$
4. $\varepsilon \cup 0(01^*1 \cup 00)^*0$
5. $\varepsilon \cup 0(10^*1 \cup 10)^*1$

Solution of Exercise 4

(a)–(1); (b)–(3); (c)–(5); (d)–(4); (e)–(2).

Before starting with the list of exercises, we recall the statement of the Pumping Lemma

Pumping Lemma: Let A be a regular language. Then the following property holds of A :

- (P) there exists $p \geq 1$ such that, for any string $w \in A$ of length $|w| \geq p$, there exists a decomposition of it of the form $w = xyz$, where $y \neq \varepsilon$, $|xy| \leq p$, and for all $i \geq 0$, the string $xy^iz \in A$.

The Pumping Lemma is often used to prove that a language A is nonregular, by using its (equivalent) contrapositive form:

Pumping Lemma (contrapositive form): Let A be a language and suppose that the following property holds of A :

- ($\neg P$) for all $p \geq 1$, there exists a string $w \in A$ of length $|w| \geq p$ such that, for all its decompositions of the form $w = xyz$, where $y \neq \varepsilon$ and $|xy| \leq p$, there exists $i \geq 0$ such that the string $xy^iz \notin A$.

Then, A is nonregular.

So, to prove A is nonregular it is sufficient to show that the property ($\neg P$) holds of A . Note that, in the slides we also saw an equivalent *game characterisation* of the statement above. You can also use this alternative way of interpreting the Pumping Lemma to solve the exercises.

Exercise 1*

Prove that the language $L_1 = \{ab^n c^n \mid n \geq 0\}$ is nonregular.

Solution of Exercise 1

To show that L_1 is nonregular, we use the game characterisation of the Pumping Lemma and provide a universal winning strategy against the demon.

1. The demon picks $p \geq 1$
2. A good response is to pick $w = ab^p c^p$. (Note that, so far we followed the rules of the game because $w \in L_1$ and $|w| = 2p + 1 \geq p$).
3. The demon picks x, y, z such that $w = xyz$, $y \neq \varepsilon$ and $|xy| \leq p$.
4. No matter what the demon picked we can always find $i \geq 0$ such that $xy^iz \notin L_1$.
Note that by the condition that $|xy| \leq p$, we have that $xy = ab^j$ for some $0 \leq j < p$ and $z = b^{(p-j)} c^p$. We choose i according to the following cases:

Case $x = \varepsilon$: Then $y = ab^j$. Thus, by picking $i = 0$, we get $xy^0 z = z = b^{(p-j)} c^p \notin L_1$, because it does not start with the symbol a

Case $x \neq \varepsilon$: Then y contains only b 's. Thus, by picking $i = 2$, we get that the string $xy^i z = ab^{p+|y|} c^p \notin L_1$, because as $y \neq \varepsilon$, $p + |y| > p$.

Exercise 2*

Prove that the language $L_2 = \{0^n 1^m 0^n \mid n, m \geq 0\}$ is nonregular.

Solution of Exercise 2

To show that L_2 is nonregular, we use the game characterisation of the Pumping Lemma and provide a universal winning strategy against the demon.

1. The demon picks $p \geq 1$
2. A good response is to pick $w = 0^p 1 0^p$. (Note that, so far we followed the rules of the game because $w \in L_2$ and $|w| = 2p + 1 \geq p$).
3. The demon picks x, y, z such that $w = xyz$, $y \neq \varepsilon$ and $|xy| \leq p$.
4. No matter what the demon picked by picking $i = 2$ we win.
Indeed, by the condition that $|xy| \leq p$ we have that y contains only 0's. Thus, $xy^2z = 0^{p+|y|} 1 0^p \notin L_2$, because as $y \neq \varepsilon$, we have that $p + |y| > p$.

Exercise 3*

Prove that the language $L_3 = \{a^n b^m \mid n, m \geq 0 \text{ and } n \neq m\}$ is nonregular.

Solution of Exercise 3

This time it is convenient to simplify our task by recalling that regular languages are closed under complement and intersection. Note that

$$\overline{L_3} \cap L(a^* b^*) = \{a^n b^n \mid n \geq 0\},$$

where $\overline{L_3}$ denotes the complement of L_3 . Therefore, if we assume for a contradiction that L_3 were regular, then also $\{a^n b^n \mid n \geq 0\}$ would be regular, which we already know it isn't (see Example 1.73 in M. Sipser, *Introduction to the Theory of Computation*, 3ed.).

For completeness, below we also propose an alternative solution using the game characterisation of the Pumping Lemma by providing a universal winning strategy against the demon.

1. The demon picks $p \geq 1$
2. A good response is to pick $w = a^{p!} b^{2p!}$. (Note that, so far we followed the rules of the game because $w \in L_3$ (indeed $p! \neq 2p!$) and $|w| = 3p! \geq p$).
3. The demon picks x, y, z such that $w = xyz$, $y \neq \varepsilon$ and $|xy| \leq p$.

4. No matter what the demon picked we can always find $i \geq 0$ such that $xy^iz \notin L_3$.

Observe that by $|xy| \leq p$ and the fact that $p \leq p!$, the string y must be formed only by a 's, that is, $y = a^k$ for some $0 < k \leq p$ (k is nonzero, because $y \neq \varepsilon$). Moreover,

$$xy^iz = a^{p!+(i-1)k}b^{2p!}.$$

The player You picks $i = \frac{p!}{k} + 1$. This is a good choice for two reasons:

- (a) i is an integer strictly greater than 1 (indeed, as $k \leq p$, $p!$ is divisible by k);
- (b) moreover, for such a choice of i , we have that $p! + (i-1)k = 2p!$.

Since we observed that $xy^iz = a^{p!+(i-1)k}b^{2p!}$, by (4b) we can infer that $xy^iz \notin L_3$. Thus, the player You wins the game.

Exercise 4

Prove that the language $L_4 = \{a^ib^jc^k \mid i, j, k \geq 0 \text{ and if } i = 1 \text{ then } j = k\}$ is nonregular.

Solution of Exercise 4

Also this time we simplify our task by using the closure properties of regular languages. Assume for a contradiction that L_4 were regular. Then also the language

$$L_4 \cap L(ab^*c^*) = \{ab^nc^n \mid n \geq 0\}$$

would be regular. However, as we already showed in Exercise 1 the above is nonregular. Thus, the only possibility is that L_4 is nonregular.

Exercise 5 (optional)

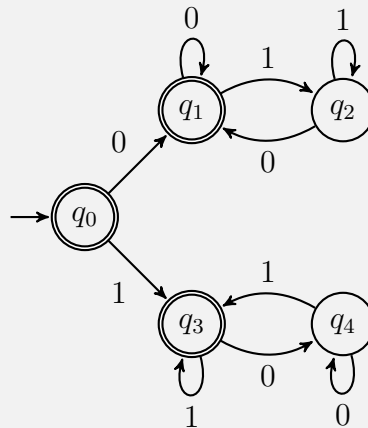
In M. Sipser, *Introduction to the Theory of Computation*, 3ed., it is claimed that the language

$$D = \{w \mid w \text{ has an equal number of occurrences of } 01 \text{ and } 10 \text{ as substrings}\}.$$

is regular. This is a bit surprising as at first glance it seems to match the typical form of a nonregular language (its definition requires one to count the occurrences of certain substrings!). Prove or disprove the claim (i.e., if you believe that D is regular provide a finite automaton recognising it; otherwise try to use the Pumping Lemma to show that D is nonregular).

Solution of Exercise 5

The language D is indeed regular. Before providing the finite automaton recognising D notice that the strings $\varepsilon, 0, 1, 010, 0000010, 101, 1111101, 1111100001$ are all elements of D . In particular, the only thing we need to remember while processing an input string, is that: if you read a 1 after a 0, you accept only after eventually reading another 0; and dually, if you read a 0 after a 1, you accept only after eventually reading another 1.



Exercise 6 (optional)

Consider the following alternative version of the Pumping Lemma

Pumping Lemma[#]: Let A be a regular language. Then the following property holds of A :

($P^\#$) there exists $p \geq 1$ such that, for any strings u, w, v with $uvw \in A$ and $|w| \geq p$, there exist a decomposition of w of the form $w = xyz$, where $y \neq \varepsilon$, and for all $i \geq 0$, the string $uxy^izv \in A$.

- Prove the above alternative version of the Pumping Lemma
- What is the contrapositive form of the above?
- Use (b) to prove that the language L_4 from Exercise 4 is nonregular.

Solution of Exercise 6

- (a) Let $M = (Q, \Sigma, \delta, q_0, F)$ be the DFA recognising A , that is $L(M) = A$. Choose p to be the number of states of M . Then, for arbitrary strings u, w, v over Σ such that $uvw \in A$ and $|w| \leq p$, we have that the corresponding accepting sequence for uvw in M is of the form:

$$r_0 \xrightarrow{\overbrace{a_1 \dots a_k}^u} r_k \xrightarrow{\overbrace{a_{k+1} \dots a_j}^w} r_j \xrightarrow{\overbrace{a_{j+1} \dots a_n}^v} r_n,$$

where $r_0 = q_0$ (the start state), $r_n \in F$ (an accept state) and $j - k \geq p + 1$. By the pigeonhole principle, within the subsequence $r_k, \dots, r_j$ there must be a repetition of the same state. Let $h < l$ be the indices of these two (separate) occurrences. Then we can subdivide the sequence $r_k, \dots, r_j$ into three pieces as follows

$$r_k \xrightarrow{\overbrace{a_{k+1} \dots a_h}^x} r_h \xrightarrow{\overbrace{a_{h+1} \dots a_l}^y} r_l \xrightarrow{\overbrace{a_{l+1} \dots a_j}^z} r_j.$$

Note that, since $h \neq l$, the string y cannot be empty (in symbols, $y \neq \varepsilon$). Moreover, since $r_h = r_l$ are the same state, M must accept also any string of the form $uxy^i zv$, for $i \geq 0$. Indeed, an accepting computation for it in M is

$$r_0, \dots, r_k, \dots, r_h, (r_{h+1}, \dots, r_l)^i, r_{l+1}, \dots, r_j, \dots, r_n,$$

where the subsequence $r_{h+1}, \dots, r_l$ is repeated i -times. Thus, $uxy^i zv \in A$.

- (b) The contrapositive form is as follows:

Pumping Lemma[‡] (contrapositive form): Let A be a language and suppose the following property holds of A :

- ($\neg P^\sharp$) for all $p \geq 1$, there exists strings u, w, v such that $uvw \in A$, $|w| \geq p$, and for all decompositions of w of the form $w = xyz$ with $y \neq \varepsilon$, there exists $i \geq 0$ such that the string $xy^i zv \notin A$.

Then, A is nonregular.

(c) Next we use the solution of (b) to prove that the language

$$L_4 = \{a^i b^j c^k \mid i, j, k \geq 0 \text{ and if } i = 1 \text{ then } j = k\}$$

is nonregular. To do so, it is sufficient to prove the property $(\neg P^\#)$ holds of L_4 .

Let $p \geq 1$. Choose $u = a$, $w = b^p$, and $v = c^p$. Note that $uvw = ab^p c^p \in L_4$ and $|w| \geq p$. For any decomposition of w of the form $w = xyz$ with $y \neq \varepsilon$ we necessarily have that x, y, z are composed only of b 's. Therefore, by choosing $i = 2$ we have that the string $uxy^i z v = ab^{p+|y|} c^p \notin L_4$, because $p + |y| \neq p$.

As an exercise, try to use the standard version of the Pumping Lemma (without using the closure property trick) and see what are the differences with respect to this new version.

Exercise 1

Consider the following context-free grammar G

$$S \rightarrow ABS \mid AB$$

$$A \rightarrow aA \mid a$$

$$B \rightarrow bA$$

Which of the following strings are in $L(G)$ and which are not? Provide derivations for those that are in $L(G)$ and reasons for those that are not.

1. $aabaab$

3. $aabbba$

2. $aaaaba$

4. $abaaba$

Solution of Exercise 1

The language generated by G is $(a^+ba^+)^+$. Broken down in pieces, the reason is that the variable A generates the language a^+ ; the variable B generates the language ba^+ ; thus, the start variable S generates $(a^+ba^+)^+$.

1. $aabaab \notin L(G)$, because is not a string in $(a^+ba^+)^+$.
2. $aaaaba \in L(G)$, the derivation proving the membership is
 $S \Rightarrow AB \Rightarrow aAB \Rightarrow aaAB \Rightarrow aaaAB \Rightarrow aaaaB \Rightarrow aaaabA \Rightarrow aaaaba$.
3. $aabbba \notin L(G)$, because is not a string in $(a^+ba^+)^+$.
4. $abaaba \in L(G)$, the derivation proving the membership is
 $S \Rightarrow ABS \Rightarrow aBS \Rightarrow abAS \Rightarrow abaS \Rightarrow abaAB \Rightarrow abaaB \Rightarrow abaabA \Rightarrow abaaba$.

Exercise 2*

Give context-free grammars that generate the following languages over $\Sigma = \{0, 1\}$:

1. $L_1 = \{w \mid w \text{ contains at least three occurrences of } 1\}$
2. $L_2 = \{w \mid w \text{ starts and ends with the same symbol}\}$
3. $L_3 = \{w \mid \text{the length of } w \text{ is odd and its middle symbol is } 0\}$
4. $L_4 = \emptyset$
5. $L_5 = \{\varepsilon\}$

Solution of Exercise 2

1. L_1 is generated by a CFG with the following rules

$$S \rightarrow A1A1A1A \qquad A \rightarrow \varepsilon \mid \Sigma A \qquad \Sigma \rightarrow 0 \mid 1$$

2. L_2 is generated by a CFG with the following rules

$$S \rightarrow 0 \mid 1 \mid 1A1 \mid 0A0 \qquad A \rightarrow \varepsilon \mid \Sigma A \qquad \Sigma \rightarrow 0 \mid 1$$

3. L_3 is generated by a CFG with the following rules

$$S \rightarrow 0 \mid \Sigma S \Sigma \qquad \Sigma \rightarrow 0 \mid 1$$

4. Recall that strings generated by a CFG are formed only by terminal symbols. Therefore, L_4 can be generated by a CFG with rule $S \rightarrow S$.

5. L_5 is generated by a CFG with the following rule $S \rightarrow \varepsilon$.

Exercise 3*

Give context-free grammars that generate the following languages over $\Sigma = \{0, 1\}$:

1. $L_6 = \{0^n 1^m \mid n \neq m\}$
2. $L_7 = \{w \mid \text{equal occurrences of 0's and 1's}\}$
3. $L_8 = \{w \mid \text{strictly more occurrences of 0's than 1's}\}$
4. $L_9 = \{uv \mid |u| = |v| \text{ and } u \neq v\}$

Solution of Exercise 3

See the appendix for further details about the solutions.

1. L_6 is generated by a CFG with the following rules

$$\begin{aligned} S &\rightarrow A \mid B \\ A &\rightarrow 0A1 \mid 0A \mid 0 \\ B &\rightarrow 0B1 \mid B1 \mid 1 \end{aligned}$$

2. L_7 is generated by a CFG with the following rules

$$S \rightarrow \varepsilon \mid 0S1 \mid 1S0 \mid SS$$

3. L_8 is generated by a CFG with the following rules

$$S \rightarrow 0 \mid 0S \mid 0S1S \mid 1S0S$$

4. L_9 is generated by a CFG with the following rules

$$S \rightarrow AB \mid BA$$

$$A \rightarrow 0 \mid \Sigma A \Sigma$$

$$B \rightarrow 1 \mid \Sigma B \Sigma$$

$$\Sigma \rightarrow 0 \mid 1$$

Exercise 4

Consider the following context-free grammar G with start variable A .

$$A \rightarrow XAX \mid S$$

$$S \rightarrow aTb \mid bTa$$

$$T \rightarrow XTX \mid X \mid \varepsilon$$

$$X \rightarrow a \mid b$$

1. Provide a formal description of G by defining all its components;
2. Give five strings in $L(G)$;
3. Give five strings not in $L(G)$;
4. Which of the following is a derivation in G ?

$$(a) \quad T \Rightarrow aba$$

$$(d) \quad T \Rightarrow^* XX$$

$$(b) \quad T \Rightarrow^* aba$$

$$(e) \quad T \Rightarrow^* XXX$$

$$(c) \quad XXX \Rightarrow^* aba$$

$$(f) \quad S \Rightarrow^* \varepsilon$$

Solution of Exercise 4

1. The grammar G is formally described by the tuple (V, Σ, R, A) , where

$$V = \{A, X, S, T\} \quad \Sigma = \{a, b\}$$

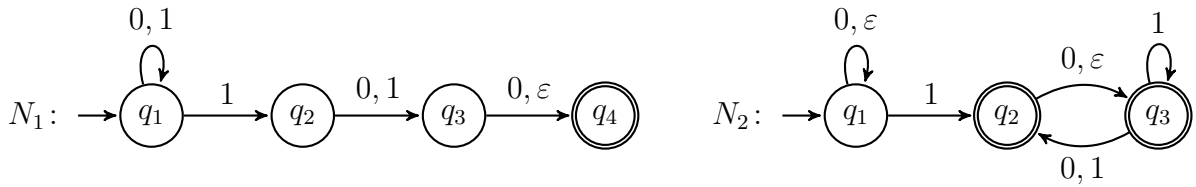
$$R = \{(A, XAX), (A, S), (S, aTb), (S, bTa), (T, XTX), (T, X), (T, \varepsilon), (X, a), (X, b)\}$$

2. $\{ab, aaabb, abaa, ba, bbab\} \subseteq L(G)$.

3. Let $\Sigma = \{a, b\}$. One can note that $L(G) = \{\Sigma^n x \Sigma^* y \Sigma^n \mid x \neq y, n \geq 0\} \subseteq \Sigma^*$. Therefore, $\{\varepsilon, a, b, aa, bb\} \cap L(G) = \emptyset$.
4. (a) $T \not\Rightarrow aba$
 (b) $T \Rightarrow XTX \Rightarrow aTX \Rightarrow aXX \Rightarrow abX \Rightarrow aba$
 (c) $XXX \Rightarrow aXX \Rightarrow abX \Rightarrow aba$
 (d) $T \Rightarrow XTX \Rightarrow XX$
 (e) $T \Rightarrow XTX \Rightarrow XXX$
 (f) $S \not\Rightarrow^* \varepsilon$

Exercise 5

Provide context-free grammars equivalent to the following NFAs.



Solution of Exercise 5

In both cases, we apply the conversion algorithm to turn a finite automaton into a CFG.

1. A context-free grammar equivalent to N_1 is the following, with Q_1 as start variable

$$\begin{aligned} Q_1 &\rightarrow 0Q_1 \mid 1Q_1 \mid 1Q_2 \\ Q_2 &\rightarrow 0Q_3 \mid 1Q_3 \\ Q_3 &\rightarrow 0Q_4 \mid Q_4 \\ Q_4 &\rightarrow \varepsilon \end{aligned}$$

2. A context-free grammar equivalent to N_2 is the following, with Q_1 as start variable

$$\begin{aligned} Q_1 &\rightarrow 0Q_1 \mid Q_1 \mid 1Q_2 \\ Q_2 &\rightarrow 0Q_3 \mid \varepsilon Q_3 \mid \varepsilon \\ Q_3 &\rightarrow 0Q_2 \mid 1Q_2 \mid 1Q_3 \mid \varepsilon \end{aligned}$$

Exercise 6

Provide context-free grammars equivalent to the following regular expressions

1. 0^*10^*
2. $1 \cup 0^*\emptyset^*$
3. $(01^+)^+$

Solution of Exercise 6

1. A grammar for 0^*10^* can be

$$\begin{aligned} S &\rightarrow Z 1 Z \\ Z &\rightarrow \varepsilon \mid 0 Z \end{aligned}$$

2. A grammar for $1 \cup 0^*\emptyset^* = 1 \cup 0^*$ can be

$$\begin{aligned} S &\rightarrow 1 \mid Z \\ Z &\rightarrow \varepsilon \mid 0 Z \end{aligned}$$

3. A grammar for $(01^+)^+$ can be

$$\begin{aligned} S &\rightarrow A \mid A S \\ A &\rightarrow 0 B \\ B &\rightarrow 1 \mid 1 B \end{aligned}$$

Exercise 1*

Give pushdown automata for each of the following languages over $\Sigma = \{0, 1\}$.

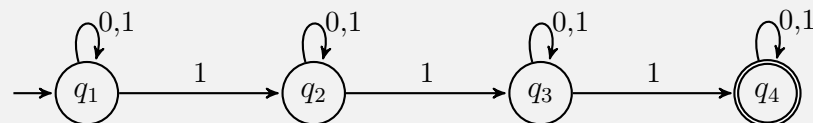
$$L_1 = \{w \mid w \text{ contains at least three occurrences of } 1\text{s}\}$$

$$L_2 = \{w \mid w \text{ starts and ends with the same symbol}\}$$

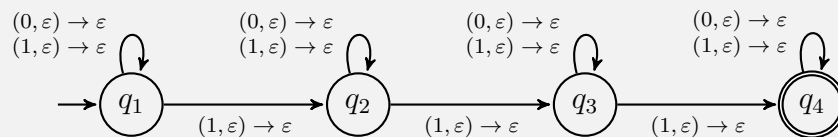
$$L_3 = \{w \mid |w| \text{ is odd and } 0 \text{ is its middle symbol}\}$$

Solution of Exercise 1

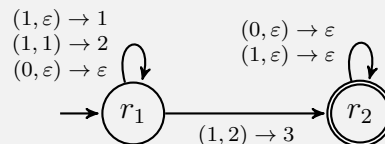
1. L_1 is regular and the following NFA recognizes it



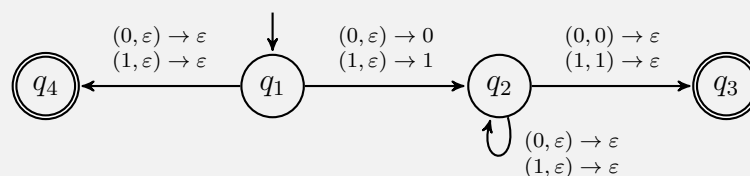
The above DFA is encoded into a PDA as follows



One can alternatively use the stack to “count” the number of 1s: after consuming the first 3 ones, the PDA moves into the accepting state. This idea is implemented by the following automaton:



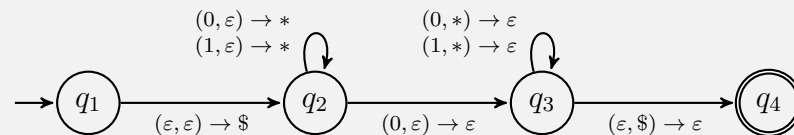
2. Also L_2 is regular, so we could give a PDA from its corresponding DFA as done above. However, we can obtain a smaller PDA by storing the first symbol in the top of the stack which can be used later to check if it coincides with last symbol of the string. This idea is implemented by the following automaton:



3. Differently from the previous cases, L_3 is not regular. Indeed, by assuming, for the sake of finding a contradiction, that L_3 is regular we obtain that the language

$$L_3 \cap L(1^*01^*) = \{1^n01^n \mid n \in \mathbb{N}\}$$

is regular. But in a previous exercise session we have seen that it is not the case. A PDA that recognizes L_3 is the following.



The above PDA prepares the stack by pushing symbol \$ to mark the end of the stack. Then, for each symbol that precedes the middle symbol of the string, a stack symbol * is pushed on the stack. After consuming the middle symbol, the PDA removes one by one all the * symbols collected in the stack until the symbol \$ is found. In this way the PDA can check that the number of symbols following the 0 in the middle is the same of those that precede it.

Exercise 2*

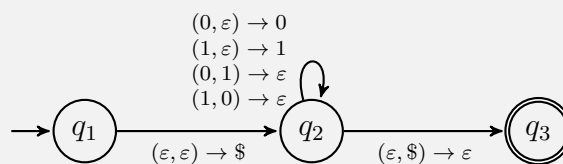
Give pushdown automata for each of the following languages over $\Sigma = \{0, 1\}$.

$$L_4 = \{w \mid w \text{ has equal occurrences of 0's and 1's}\}$$

$$L_5 = \{w \mid w \text{ has strictly more occurrences of 0's than 1's}\}$$

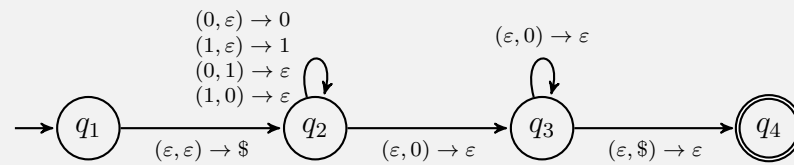
Solution of Exercise 2

1. A PDA that recognizes L_4 is the following.



The above PDA prepares the stack by first pushing symbol \$ to mark the end of the stack. Then, nondeterministically, it chooses either to push the input symbols in the stack or to pop the corresponding opposite symbol from the stack. The PDA accepts only if the stack is left empty, that is, when the symbol \$ is the only one left.

1. A PDA that recognizes L_5 is the following.



The above PDA works similarly to the above. Acceptance, however can happen only when the stack is left only with 0's and with last symbol \$, marking the end of the stack.

Exercise 3*

Convert each of the CFGs below into an equivalent PDA.

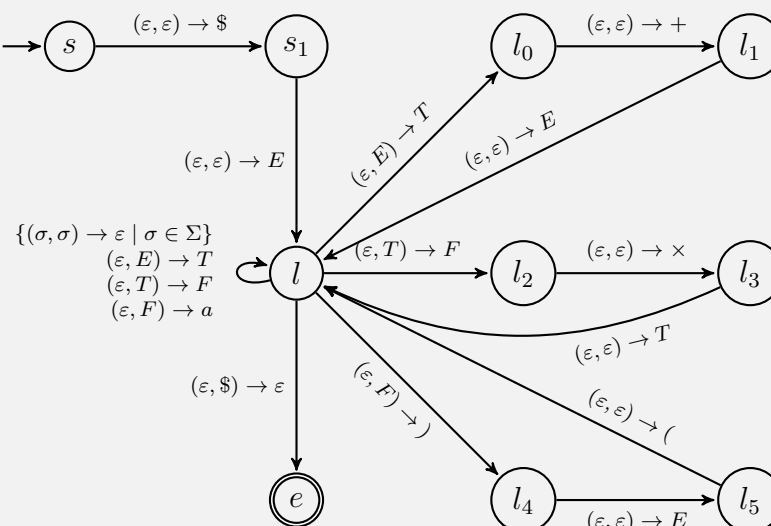
$$\begin{aligned} G_1: E &\rightarrow E + T \mid T \\ T &\rightarrow T \times F \mid F \\ F &\rightarrow (E) \mid a \end{aligned}$$

$$\begin{aligned} G_2: R &\rightarrow XRX \mid S \\ S &\rightarrow aTb \mid bTa \\ T &\rightarrow XTX \mid X \mid \varepsilon \\ X &\rightarrow a \mid b \mid \varepsilon \end{aligned}$$

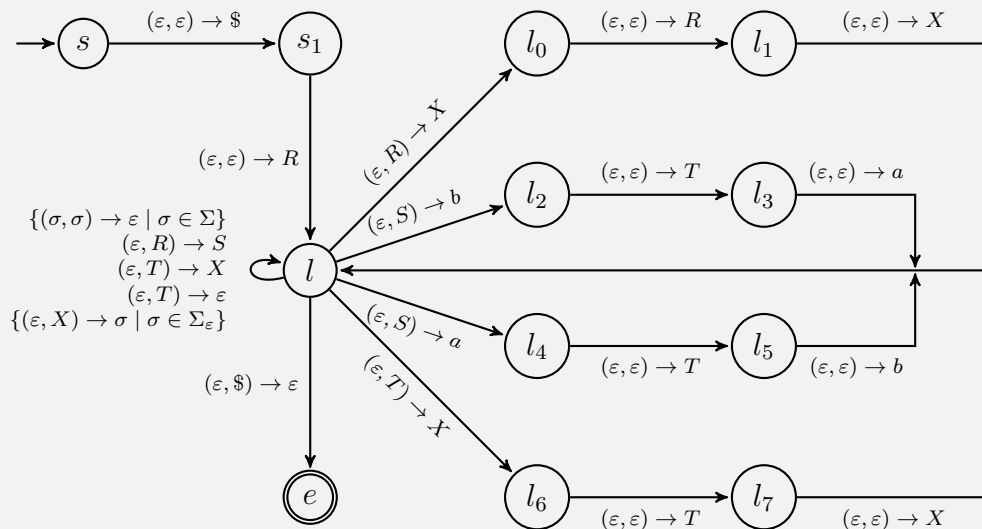
Follow the general conversion technique described in Lemma 2.21, M. Sipser, *Introduction to the Theory of Computation*, 3ed.

Solution of Exercise 3

1. A PDA that recognizes the language generated by G_1 is the following

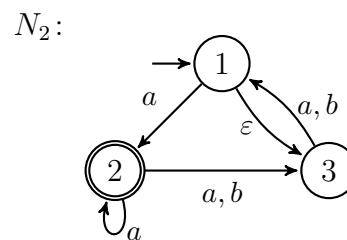
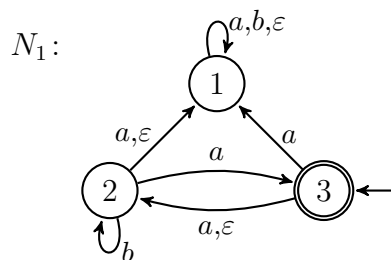


2. A PDA that recognizes the language generated by G_2 is the following

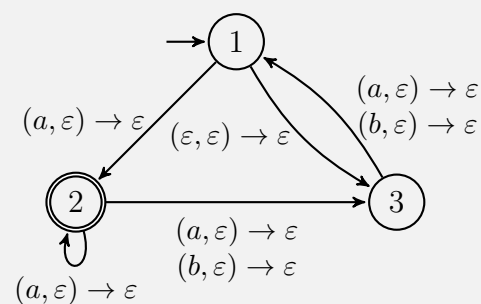
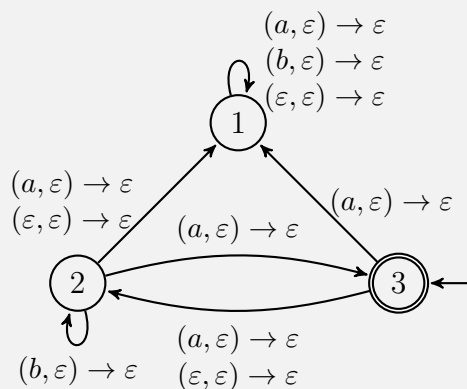


Exercise 4

Construct an equivalent PDA for each of the following NFAs.



Solution of Exercise 4



Exercise 5 (optional)

Convert the the grammar G_1 from Exercise 3 in Chomsky normal form.

Solution of Exercise 5

To convert G_1 into Chomsky normal form we first introduce a new start variable (step 1):

$$\begin{aligned} S &\rightarrow E \\ E &\rightarrow E + T \mid T \\ T &\rightarrow T \times F \mid F \\ F &\rightarrow (E) \mid a \end{aligned}$$

There are no ε -rules to remove (step 2), so next we remove unitary rules (step 3):

$$\begin{array}{lll} S \rightarrow E & S \rightarrow E & S \rightarrow E + T \mid T \times F \mid (E) \mid a \\ E \rightarrow E + T \mid T & E \rightarrow E + T \mid T \times F \mid (E) \mid a & E \rightarrow E + T \mid T \times F \mid (E) \mid a \\ T \rightarrow T \times F \mid (E) \mid a & T \rightarrow T \times F \mid (E) \mid a & T \rightarrow T \times F \mid (E) \mid a \\ F \rightarrow (E) \mid a & F \rightarrow (E) \mid a & F \rightarrow (E) \mid a \end{array}$$

In (step 4) we replace rules of type $A \rightarrow u_1, u_2 \dots u_k$, where $k > 2$. First we make a new rule for $+ T$ (left); then a rule for $\times F$ (mid); finally, A rule for E) (right)

$$\begin{array}{lll} S \rightarrow EE_1 \mid TE_2 \mid (P_1 \mid a & S \rightarrow EE_1 \mid TE_2 \mid (E) \mid a & S \rightarrow EE_1 \mid TE_2 \mid (P_1 \mid a \\ E \rightarrow EE_1 \mid TE_2 \mid (E) \mid a & E \rightarrow EE_1 \mid TE_2 \mid (E) \mid a & E \rightarrow EE_1 \mid TE_2 \mid (P_1 \mid a \\ T \rightarrow TE_2 \mid (P_1 \mid a & T \rightarrow TE_2 \mid (E) \mid a & T \rightarrow TE_2 \mid (P_1 \mid a \\ F \rightarrow (P_1 \mid a & F \rightarrow (E) \mid a & F \rightarrow (P_1 \mid a \\ E_1 \rightarrow + T & E_1 \rightarrow + T & E_1 \rightarrow + T \\ E_2 \rightarrow \times F & E_2 \rightarrow \times F & E_2 \rightarrow \times F \\ E_1 \rightarrow + T & E_1 \rightarrow + T & E_1 \rightarrow + T \\ E_2 \rightarrow \times F & E_2 \rightarrow \times F & E_2 \rightarrow \times F \\ P_1 \rightarrow E & P_1 \rightarrow E & P_1 \rightarrow E \end{array}$$

In step 5 we eliminate rules of type $A \rightarrow uB$, replacing them with $A \rightarrow UB$ and $U \rightarrow u$ (multiple rewrites shown at once here):

$$\begin{array}{ll} S \rightarrow EE_1 \mid TE_2 \mid P_2P_1 \mid a & P_1 \rightarrow EP_3 \\ E \rightarrow EE_1 \mid TE_2 \mid P_2P_1 \mid a & P_2 \rightarrow (\\ T \rightarrow TE_2 \mid P_2P_1 \mid a & P_3 \rightarrow) \\ F \rightarrow P_2P_1 \mid a & P_3 \rightarrow) \\ E_1 \rightarrow AT & M \rightarrow \times \\ E_2 \rightarrow MF & A \rightarrow + \end{array}$$

Exercise 1*

Consider the following three languages over the alphabet $\Sigma = \{a, b, c\}$

$$L_1 = \{a^i b^j c^k \mid i, j, k \geq 0, i = j \leq k\}$$

$$L_2 = \{a^i b^j c^k \mid i, j, k \geq 0, i = j\}$$

$$L_3 = \{a^i b^j c^k \mid i, j, k \geq 0, j \leq k\}$$

Which of the above languages is context-free and which is not? Justify your answer by providing formal proofs of your claims. (Hint: only two of the above are context-free)

Solution of Exercise 1

L_2 and L_3 are context-free languages, and are generated by the following grammars

$$L_2: S \rightarrow TC$$

$$C \rightarrow \varepsilon \mid cC$$

$$T \rightarrow \varepsilon \mid aTb$$

$$L_3: S \rightarrow AT$$

$$A \rightarrow \varepsilon \mid aA$$

$$T \rightarrow \varepsilon \mid Tc \mid bTc$$

L_1 is not context-free. To show that L_1 is not context-free, we use the game characterisation of the Pumping Lemma and provide a universal winning strategy against the demon.

1. The demon picks $p \geq 0$
2. A good response is to pick $w = a^p b^p c^p$. (So far we followed the rules of the game as $w \in L_1$ and $|w| = 4p \geq p$).
3. The demon picks u, v, x, y, z such that $w = uvxyz$, $vy \neq \varepsilon$ and $|vxy| \leq p$.
4. No matter what the demon picked we can always win by choosing an adequate $i \geq 0$. To show that this is indeed the case, notice that, by $|vxy| \leq p$, the substring vxy cannot contain both a 's and c 's since this would imply $|ab^p c| \leq p$. Therefore, we have the following cases:

Case 1 vxy has only one symbol: Thus, $vxy = \sigma^k$ for some $\sigma \in \Sigma$ and $k \leq p$. Then we have two sub-cases:

- (a) $\sigma \in \{a, b\}$: $uv^0 xy^0 z \notin L_1$ because it has different number of a 's and b 's
- (b) $\sigma = c$: $uv^0 xy^0 z \notin L_1$ because it has less c 's than a 's and b 's.

Case 2 vxy has both a 's and b 's: $uv^2 xy^2 z \notin L_1$ as it has more a 's or b 's than c 's.

Case 3 vxy has both b 's and c 's: $uv^0 xy^0 z \notin L_1$ as it has less b 's than a 's.

Remark: Note that $L_1 = L_2 \cap L_3$. Thus the above proves that context-free languages are not closed under intersection.

Exercise 2

Consider the language

$$L_4 = \{u\#w \mid u, w \in \{0, 1\}^*, u \text{ is a prefix of } w\}$$

Is L_4 context-free? Justify your answer by providing a formal proof of your claim.

Solution of Exercise 2

L_4 is not context-free. As a proof to our claim we provide a universal winning strategy against the demon in the game characterisation of the Pumping Lemma.

1. The demon picks $p \geq 0$
2. A good response is to pick $w = 0^p 1^p \# 0^p 1^p$. (So far we followed the rules of the game as $w \in L_4$ and $|w| = 4p + 1 \geq p$).
3. The demon picks u, v, x, y, z such that $w = uvxyz$, $vy \neq \varepsilon$ and $|vxy| \leq p$.
4. No matter what the demon picked we can always win by choosing an adequate $i \geq 0$ for each subdivision of w into $uvxyz$:

Case 1 v or y contains $\#$: Then $uv^0xy^0z \notin L_4$ because has no symbol $\#$.

Case 2 v and y are both to the left of $\#$: Then $uv^2xy^2z \notin L_4$ because the substring to the left of $\#$ is longer than that to the right, therefore the former cannot be a prefix of the latter.

Case 3 v and y are both to the right of $\#$: Then $uv^0xy^0z \notin L_4$ because the substring to the right of $\#$ is shorter than that to the left, therefore the latter cannot be a prefix of the former.

Case 4 v and y are respectively to the left and to the right of $\#$: By $|vxy| \leq p$ and $vy \neq \varepsilon$, v has only 1's and y has only 0's. Thus, $uv^2xy^2z \notin L_4$ because the substring to the left of $\#$ has more 1's than that to the right or the substring to the right has more 0's than that to the left.

Exercise 3*

Here is a wrong attempt to show that a language is not context-free.

Consider the language

$$L_5 = \{uu \mid u \in \{a, b\}^*\}.$$

We use the pumping lemma to show that L_5 is not context-free. Choose $w = aabbaabb$. So we can split it into $u = aa$, $v = b$, $x = baa$, $y = bb$ and $z = \varepsilon$. But then we have that $uv^2xy^2z \notin L_5$.

Explain the main reasons why the above attempt of proof is wrong.

Solution of Exercise 3

Some reasons are the following

- no mentioning of the fact that L_5 is assumed to be context-free. This is crucial if you are looking for a contradiction in a proof by contradiction.
- no pumping length p is mentioned, so the condition $|w| \geq p$ may not hold.
- only a single split is considered, rather than studying all possible splits of w of the form $uvxyz$.
- $|vxy| \leq p$ on this particular split may not hold.

Exercise 4 (optional)

Prove that context-free languages are closed under intersection with regular languages. (that is, if A is a CFL and B is a regular language, then $A \cap B$ is a CFL).

Solution of Exercise 4

Let $N_1 = (Q_1, \Sigma, \delta_1, q_0^1, F_1)$ be an arbitrary NFA and $P_2 = (Q_2, \Sigma, \Gamma, \delta_2, q_0^2, F_2)$ an arbitrary PDA. We assume without loss of generality that N_1, P_2 are over the same alphabet Σ and that each state has a ε -loop, that is, $\forall q \in Q_1. q \in \delta_1(q, \varepsilon)$ and $\forall r \in Q_2. (r, \varepsilon) \in \delta_2(r, \varepsilon, \varepsilon)$. Define the PDA $P = (Q, \Sigma, \Gamma, q_0, \delta, F)$, where

$$Q = Q_1 \times Q_2, \quad q_0 = (q_0^1, q_0^2), \quad F = F_1 \times F_2, \quad \delta: Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \rightarrow \mathcal{P}(Q \times \Gamma_\varepsilon),$$

where

$$\delta((q_1, q_2), a, x) = \{((p_1, p_2), y) \mid p_1 \in \delta_1(q_1, a) \text{ and } (p_2, a, y) \in \delta_2(q_2, a, x)\}$$

Next we prove $L(P) = L(N_1) \cap L(P_2)$.

($\subseteq$) Let $w \in L(P)$ and $(q_0, r_0) \mid s_0 \xrightarrow{w_1} (q_1, r_1) \mid s_1 \xrightarrow{w_2} \dots \xrightarrow{w_m} (q_m, r_m) \mid s_m$ be an accepting computation for $w = w_1 \dots w_m$ in P , where $w_i \in \Sigma_\varepsilon$. By construction, we have that $(q_m, r_m) \in F = F_1 \times F_2$, therefore $q_m \in F_1$ and $r_m \in F_2$. Moreover, $q_0 = q_0^1$ and $r_0 = q_0^2$. By definition of δ , for all $j \in \{0, \dots, m-1\}$, it holds that $q_{j+1} = \delta_1(q_j, w_{j+1})$ and $(r_{j+1}, y) \in \delta_2(r_j, w_{j+1}, x)$, for $s_j = xt$ and $s_{j+1} = yt$. Since $q_0, \dots, q_m$ and $r_0 \mid s_0, \dots, r_m \mid s_m$ are accepting computations in N_1 and P_2 , respectively, we get $w \in L(N_1) \cap L(P_2)$.

($\supseteq$) Consider $w \in L(N_1) \cap L(P_2)$. Then, $w \in L(N_1)$ and $w \in L(P_2)$ and there exist accepting computations $q_0, \dots, q_m$ and $r_0|s_0, \dots, r_m|s_m$ for w in N_1 and P_2 , respectively. By construction, the sequence $(q_0, r_0)|s_0, \dots, (q_m, r_m)|s_m$ is accepting w in P . Thus, $w \in L(P)$.

Exercise 1*

Evaluate the following expressions and describe their derivation trees –use the big-step semantics of $\mathbb{A}\text{exp}$.

1. $(\underline{3} + \underline{12}) * (\underline{4} * (\underline{5} * \underline{8}))$
2. $(\underline{3} + (\underline{12} * \underline{4})) * (\underline{5} * \underline{8})$
3. $(\underline{3} + (\underline{12})) * ((\underline{4}) * (\underline{5} * \underline{8}))$

Solution of Exercise 1

1. Here is a derivation tree for $(\underline{3} + \underline{12}) * (\underline{4} * (\underline{5} * \underline{8}))$

$$\begin{array}{c}
 \begin{array}{c}
 \text{[MULT}_{BS}] \frac{\underline{5} \rightarrow 5 \quad \underline{8} \rightarrow 8}{\underline{5} * \underline{8} \rightarrow 40} \\
 \text{[PARENT}_{BS}] \frac{\underline{5} * \underline{8} \rightarrow 40}{(\underline{5} * \underline{8}) \rightarrow 40} \\
 \text{[MULT}_{BS}] \frac{\underline{4} \rightarrow 4}{\underline{4} * (\underline{5} * \underline{8}) \rightarrow 160} \\
 \text{[PARENT}_{BS}] \frac{\underline{4} * (\underline{5} * \underline{8}) \rightarrow 160}{(\underline{4} * (\underline{5} * \underline{8})) \rightarrow 160}
 \end{array} \\
 \text{[PLUS}_{BS}] \frac{\underline{3} \rightarrow 3 \quad \underline{12} \rightarrow 12}{\underline{3} + \underline{12} \rightarrow 15} \\
 \text{[PARENT}_{BS}] \frac{\underline{3} + \underline{12} \rightarrow 15}{(\underline{3} + \underline{12}) \rightarrow 15} \\
 \text{[MULT}_{BS}] \frac{(\underline{3} + \underline{12}) \rightarrow 15 \quad (\underline{4} * (\underline{5} * \underline{8})) \rightarrow 160}{(\underline{3} + \underline{12}) * (\underline{4} * (\underline{5} * \underline{8})) \rightarrow 2400}
 \end{array}$$

2. Here is a derivation tree for $(\underline{3} + (\underline{12} * \underline{4})) * (\underline{5} * \underline{8})$

$$\begin{array}{c}
 \begin{array}{c}
 \text{[MULT}_{BS}] \frac{\underline{12} \rightarrow 12 \quad \underline{4} \rightarrow 4}{\underline{12} * \underline{4} \rightarrow 48} \\
 \text{[PARENT}_{BS}] \frac{\underline{12} * \underline{4} \rightarrow 48}{(\underline{12} * \underline{4}) \rightarrow 48} \\
 \text{[PLUS}_{BS}] \frac{\underline{3} \rightarrow 3}{\underline{3} + (\underline{12} * \underline{4}) \rightarrow 51} \\
 \text{[PARENT}_{BS}] \frac{\underline{3} + (\underline{12} * \underline{4}) \rightarrow 51}{(\underline{3} + (\underline{12} * \underline{4})) \rightarrow 51}
 \end{array} \\
 \text{[MULT}_{BS}] \frac{(\underline{3} + (\underline{12} * \underline{4})) \rightarrow 51 \quad (\underline{5} * \underline{8}) \rightarrow 40}{(\underline{3} + (\underline{12} * \underline{4})) * (\underline{5} * \underline{8}) \rightarrow 2040}
 \end{array}$$

where proof tree for the subexpression $(\underline{5} * \underline{8})$ is the same as in (1).

3. Here is a derivation tree for $(\underline{3} + (\underline{12})) * ((\underline{4}) * (\underline{5} * \underline{8}))$

$$\begin{array}{c}
 \begin{array}{c}
 \text{[PLUS}_{BS}] \frac{\underline{3} \rightarrow 3}{\underline{3} + (\underline{12}) \rightarrow 15} \\
 \text{[PARENT}_{BS}] \frac{\underline{3} + (\underline{12}) \rightarrow 15}{(\underline{3} + (\underline{12})) \rightarrow 15} \\
 \text{[MULT}_{BS}] \frac{(\underline{3} + (\underline{12})) \rightarrow 15 \quad ((\underline{4}) * (\underline{5} * \underline{8})) \rightarrow 160}{(\underline{3} + (\underline{12})) * ((\underline{4}) * (\underline{5} * \underline{8})) \rightarrow 2400}
 \end{array} \\
 \begin{array}{c}
 \text{[PARENT}_{BS}] \frac{\underline{12} \rightarrow 12}{(\underline{12}) \rightarrow 12} \\
 \text{[PARENT}_{BS}] \frac{\underline{4} \rightarrow 4}{(\underline{4}) \rightarrow 4} \\
 \text{[PARENT}_{BS}] \frac{\underline{5} * \underline{8} \rightarrow 40}{(\underline{5} * \underline{8}) \rightarrow 40} \\
 \text{[MULT}_{BS}] \frac{(\underline{4}) * (\underline{5} * \underline{8}) \rightarrow 160}{((\underline{4}) * (\underline{5} * \underline{8})) \rightarrow 160}
 \end{array}
 \end{array}$$

Again, the proof tree for the subexpression $(\underline{5} * \underline{8})$ is the same as in (1).

Exercise 2

Suggest a new small-step semantics for $\mathbb{Aexp}$, which is deterministic.
(Hint: Use rules to ensure that the evaluation is done from left to right.)

Solution of Exercise 2

Consider the small-step semantics for $\mathbb{Aexp}$ seen in class. It can be turned into a deterministic semantics by replacing the rules $[\text{PLUS-2}_{ss}]$, $[\text{MULT-2}_{ss}]$ and, $[\text{SUB-2}_{ss}]$ with the following.

$$\begin{array}{c} [\text{PLUS-2}_{ss}] \frac{a_2 \Rightarrow a'_2}{v + a_2 \Rightarrow v + a'_2} \quad [\text{MULT-2}_{ss}] \frac{a_2 \Rightarrow a'_2}{v * a_2 \Rightarrow v * a'_2} \\ [\text{SUB-2}_{ss}] \frac{a_2 \Rightarrow a'_2}{v - a_2 \Rightarrow v - a'_2} \end{array}$$

Intuitively, these rules can be applied only when the left hand side of the expression is fully evaluated (i.e., it's a normal form). This is known as the leftmost selection strategy.

Exercise 3*

Give a big-step and a small-step semantics for $\mathbb{Bexp}$

$$b ::= a_1 = a_2 \mid a_1 < a_2 \mid \neg b_1 \mid b_1 \wedge b_2$$

assuming that we already have a big-step and a small-step semantics for $\mathbb{Aexp}$ defined by $\rightarrow_A$ and $\Rightarrow_A$ respectively.

Solution of Exercise 3

The big-step operational semantics for $\mathbb{Bexp}$ can be defined as

$$\begin{array}{c} [\text{EQ}_{tt}] \frac{a_1 \rightarrow_A v \quad a_2 \rightarrow_A v}{a_1 = a_2 \rightarrow_B tt} \quad [\text{EQ}_{ff}] \frac{a_1 \rightarrow_A v_1 \quad a_2 \rightarrow_A v_2}{a_1 = a_2 \rightarrow_B ff} \text{ if } v_1 \neq v_2 \\ [\text{LT}_{tt}] \frac{a_1 \rightarrow_A v_1 \quad a_2 \rightarrow_A v_2}{a_1 < a_2 \rightarrow_B tt} \text{ if } v_1 < v_2 \quad [\text{LT}_{ff}] \frac{a_1 \rightarrow_A v_1 \quad a_2 \rightarrow_A v_2}{a_1 < a_2 \rightarrow_B ff} \text{ if } v_1 \geq v_2 \\ [\text{NEG}_{tt}] \frac{b \rightarrow_B ff}{\neg b \rightarrow_B tt} \quad [\text{NEG}_{ff}] \frac{b \rightarrow_B tt}{\neg b \rightarrow_B ff} \\ [\text{AND}_{tt}] \frac{b_1 \rightarrow_B tt \quad b_2 \rightarrow_B tt}{b_1 \wedge b_2 \rightarrow_B tt} \\ [\text{AND1}_{ff}] \frac{b_1 \rightarrow_B ff}{b_1 \wedge b_2 \rightarrow_B ff} \quad [\text{AND2}_{ff}] \frac{b_2 \rightarrow_B ff}{b_1 \wedge b_2 \rightarrow_B ff} \end{array}$$

Notice that the above semantics is nondeterministic. Indeed if both b_1 and b_2 evaluate to ff the expression $b_1 \wedge b_2$ can be applied both to $AND1_{ff}$ and $AND2_{ff}$. However, in both cases $b_1 \wedge b_2$ is evaluated to ff . An alternative semantics for $\mathbb{B}exp$ which is deterministic can be obtained from the above by replacing rule $AND2_{ff}$ with

$$[AND2_{ff}] \frac{b_1 \rightarrow_B tt \quad b_2 \rightarrow_B ff}{b_1 \wedge b_2 \rightarrow_B ff}.$$

The small-step semantics for $\mathbb{B}exp$ can be defined as

$$\begin{array}{l} [EQ-L] \frac{a_1 \Rightarrow_A a'_1}{a_1 = a_2 \Rightarrow_B a'_1 = a_2} \quad [EQ-R] \frac{a_2 \Rightarrow_A a'_2}{v_1 = a_2 \Rightarrow_B v_1 = a'_2} \\ [EQ_{tt}] \frac{}{v = v \Rightarrow_B tt} \quad [EQ_{ff}] \frac{}{v_1 = v_2 \Rightarrow_B ff} \text{ if } v_1 \neq v_2 \\ [LT-L] \frac{a_1 \Rightarrow_A a'_1}{a_1 < a_2 \Rightarrow_B a'_1 < a_2} \quad [LT-R] \frac{a_2 \Rightarrow_A a'_2}{v_1 < a_2 \Rightarrow_B v_1 < a'_2} \\ [LT_{tt}] \frac{}{v_1 < v_2 \Rightarrow_B tt} \text{ if } v_1 < v_2 \quad [LT_{ff}] \frac{}{v_1 < v_2 \Rightarrow_B ff} \text{ if } v_1 \geq v_2 \\ [NEG] \frac{b \Rightarrow_B b'}{\neg b \Rightarrow_B \neg b'} \quad [NEG_{ff}] \frac{}{\neg tt \Rightarrow_B ff} \quad [NEG_{tt}] \frac{}{\neg ff \Rightarrow_B tt} \\ [AND-L] \frac{b_1 \Rightarrow_B b'_1}{b_1 \wedge b_2 \Rightarrow_B b'_1 \wedge b_2} \quad [AND-R] \frac{b_2 \Rightarrow_B b'_2}{tt \wedge b_2 \Rightarrow_B tt \wedge b'_2} \\ [AND1_{ff}] \frac{}{ff \wedge b_2 \Rightarrow_B ff} \quad [AND2_{ff}] \frac{}{tt \wedge ff \Rightarrow_B ff} \quad [AND_{tt}] \frac{}{tt \wedge tt \Rightarrow_B tt} \end{array}$$

It is worth noticing that the above semantics is deterministic, since each binary operator evaluates its right hand side only after having completed the evaluation of the left hand side. This is known as leftmost selection strategy.

Exercise 1*

Is there a big-step transition for each of the following cases? If so, prove it.

1. $\langle \mathbf{x} := 2; (\mathbf{skip}; \mathbf{y} := 3), [x \mapsto 3, y \mapsto 5] \rangle \rightarrow ?$
2. $\langle \mathbf{if} \ \mathbf{x} < \mathbf{y} \ \mathbf{then} \ \mathbf{z} := 5 \ \mathbf{else} \ \mathbf{S}, [x \mapsto 2, y \mapsto 3, z \mapsto 5] \rangle \rightarrow ?$
where $\mathbf{S} = (\mathbf{if} \ \mathbf{x} + 1 < \mathbf{y} \ \mathbf{then} \ \mathbf{z} := 2 \ \mathbf{else} \ \mathbf{z} := 3)$
3. $\langle \mathbf{skip}; \mathbf{x} := 3; \mathbf{while} \ \mathbf{x} \leq 5 \ \mathbf{do} \ (\mathbf{x} := \mathbf{x} + 1; \mathbf{y} := 2), s \rangle \rightarrow ?$ where $s = [x \mapsto 2, y \mapsto 0]$.

Solution of Exercise 1

In the following, the rules (ASS) and (SKIP) for assignments and skip statements will be applied without explicitly mentioning their use. Also, evaluations of expression will be treated atomically as axioms without showing any details for their derivations.

1. Let $s = [x \mapsto 3, y \mapsto 5]$.

$$[\text{COMP}] \frac{\langle \mathbf{x} := 2, s \rangle \rightarrow s_1 \quad [\text{COMP}] \frac{\langle \mathbf{skip}, s_1 \rangle \rightarrow s_1 \quad \langle \mathbf{y} := 3, s_1 \rangle \rightarrow s_2}{\langle \mathbf{skip}; \mathbf{y} := 3, s_1 \rangle \rightarrow s_2}}{\langle \mathbf{x} := 2; (\mathbf{skip}; \mathbf{y} := 3), s \rangle \rightarrow s_2}$$

where $s_1 = [x \mapsto 2, y \mapsto 5]$ and $s_2 = [x \mapsto 2, y \mapsto 3]$. The derivation above proves the existence of the following big-step transition

$$\langle \mathbf{x} := 2; (\mathbf{skip}; \mathbf{y} := 3), [x \mapsto 3, y \mapsto 5] \rangle \rightarrow [x \mapsto 2, y \mapsto 3]$$

2. Let $s = [x \mapsto 2, y \mapsto 3, z \mapsto 5]$.

$$[\text{IF-tt}] \frac{\langle \mathbf{z} := 5, s \rangle \rightarrow [x \mapsto 2, y \mapsto 3, z \mapsto 5] \quad s \vdash \mathbf{x} < \mathbf{y} \rightarrow_B \text{tt}}{\langle \mathbf{if} \ \mathbf{x} < \mathbf{y} \ \mathbf{then} \ \mathbf{z} := 5 \ \mathbf{else} \ \mathbf{S}, s \rangle \rightarrow [x \mapsto 2, y \mapsto 3, z \mapsto 5]}$$

Notice that it was not necessary to evaluate $\mathbf{S}$.

3. Let $s = [x \mapsto 2, y \mapsto 0]$ and $\mathbf{W} = \mathbf{while} \ \mathbf{x} \leq 5 \ \mathbf{do} \ (\mathbf{x} := \mathbf{x} + 1; \mathbf{y} := 2)$. To fit the page limit we split the derivation in five parts:

$$[\text{COMP}] \frac{\langle \mathbf{skip}, s \rangle \rightarrow s \quad [\text{COMP}] \frac{\langle \mathbf{x} := 3, s \rangle \rightarrow s_1 \quad \frac{\vdots}{\langle \mathbf{W}, s_1 \rangle \rightarrow s_5}}{\langle \mathbf{x} := 3; \mathbf{W}, s \rangle \rightarrow s_5}}{\langle \mathbf{skip}; (\mathbf{x} := 3; \mathbf{W}), s \rangle \rightarrow s_5}$$

$$\begin{array}{c}
\text{[COMP]} \frac{\langle \mathbf{x} := \mathbf{x} + 1 \rangle, s_1 \rightarrow s_2 \quad \langle \mathbf{y} := 2, s_2 \rangle \rightarrow s_3 \quad \vdots}{\text{[COMP]} \frac{\langle \langle \mathbf{x} := \mathbf{x} + 1; \mathbf{y} := 2 \rangle, s_1 \rangle \rightarrow s_3 \quad \langle \mathbf{W}, s_3 \rangle \rightarrow s_5}{\text{[WHILE-tt]} \frac{\langle \langle \mathbf{x} := \mathbf{x} + 1; \mathbf{y} := 2 \rangle; \mathbf{W}, s_1 \rangle \rightarrow s_5 \quad s_1 \vdash \mathbf{x} \leq 5 \rightarrow_B tt}{\langle \mathbf{W}, s_1 \rangle \rightarrow s_5}}} \\
\\
\text{[COMP]} \frac{\langle \mathbf{x} := \mathbf{x} + 1 \rangle, s_3 \rightarrow s_4 \quad \langle \mathbf{y} := 2, s_4 \rangle \rightarrow s_4 \quad \vdots}{\text{[COMP]} \frac{\langle \langle \mathbf{x} := \mathbf{x} + 1; \mathbf{y} := 2 \rangle, s_3 \rangle \rightarrow s_4 \quad \langle \mathbf{W}, s_4 \rangle \rightarrow s_5}{\text{[WHILE-tt]} \frac{\langle \langle \mathbf{x} := \mathbf{x} + 1; \mathbf{y} := 2 \rangle; \mathbf{W}, s_3 \rangle \rightarrow s_5 \quad s_3 \vdash \mathbf{x} \leq 5 \rightarrow_B tt}{\langle \mathbf{W}, s_3 \rangle \rightarrow s_5}}} \\
\\
\text{[COMP]} \frac{\langle \mathbf{x} := \mathbf{x} + 1 \rangle, s_4 \rightarrow s_5 \quad \langle \mathbf{y} := 2, s_5 \rangle \rightarrow s_5 \quad \vdots}{\text{[COMP]} \frac{\langle \langle \mathbf{x} := \mathbf{x} + 1; \mathbf{y} := 2 \rangle, s_4 \rangle \rightarrow s_5 \quad \langle \mathbf{W}, s_5 \rangle \rightarrow s_5}{\text{[WHILE-tt]} \frac{\langle \langle \mathbf{x} := \mathbf{x} + 1; \mathbf{y} := 2 \rangle; \mathbf{W}, s_4 \rangle \rightarrow s_5 \quad s_4 \vdash \mathbf{x} \leq 5 \rightarrow_B tt}{\langle \mathbf{W}, s_4 \rangle \rightarrow s_5}}} \\
\\
\text{[WHILE-ff]} \frac{s_5 \vdash \mathbf{x} \leq 5 \rightarrow_B \text{ff}}{\langle \mathbf{W}, s_5 \rangle \rightarrow s_5}
\end{array}$$

where the states used above are

$$\begin{array}{lll}
s_1 = [x \mapsto 3, y \mapsto 0] & s_2 = [x \mapsto 4, y \mapsto 0] & s_3 = [x \mapsto 4, y \mapsto 2] \\
s_4 = [x \mapsto 5, y \mapsto 2] & s_5 = [x \mapsto 6, y \mapsto 2] &
\end{array}$$

Summarising, the above derivation proves that

$$\langle \text{skip}; \mathbf{x} := 3; \text{while } \mathbf{x} \leq 5 \text{ do } (\mathbf{x} := \mathbf{x} + 1; \mathbf{y} := 2), [x \mapsto 2, y \mapsto 0] \rangle \rightarrow [x \mapsto 6, y \mapsto 2]$$

Exercise 2

Prove that $\mathbf{S} = \text{while } \neg(2 < (1 + 1)) \text{ do } (\text{if } \mathbf{x} < \mathbf{x} \text{ then } \mathbf{x} := 2 \text{ else skip})$ loops forever in the small-step semantics.

(Hint: Show that there exists $k \geq 0$ such that, for any state $s \in \mathbf{States}$, $\langle \mathbf{S}, s \rangle \Rightarrow^k \langle \mathbf{S}, s \rangle$).

Solution of Exercise 2

We show that the small-step semantics for S reduces $\langle S, s \rangle$ to itself after some number of steps, meaning that the same steps will be taken over and over again without leading to a terminal configuration.

The following show that in 4 transitions $\langle S, s \rangle$ reduces to itself, i.e., $\langle S, s \rangle \Rightarrow^4 \langle S, s \rangle$

$$\begin{aligned}
 \langle S, s \rangle &\Rightarrow \langle \text{if } \neg(2 < (1 + 1)) \text{ then } (\text{if } x < x \text{ then } x := 2 \text{ else skip}; S) \text{ else skip}, s \rangle \\
 &\Rightarrow \langle \text{if } x < x \text{ then } x := 2 \text{ else skip}; S, s \rangle \\
 &\Rightarrow \langle \text{skip}; S, s \rangle \\
 &\Rightarrow \langle S, s \rangle
 \end{aligned}$$

Exercise 3*

Find all the transitions (if there are any) in the transition sequence starting from $\langle S, s \rangle$ in the small-step semantics, for each of the following cases:

1. $S = \text{while } x \geq y \text{ do } (x := x - 1; y := y + 1) \text{ and } s = [x \mapsto 3, y \mapsto 0]$.
2. $S = \text{if } (x \leq 3 \vee y > 2) \text{ then } (x := x + 3; y := 2) \text{ else skip and } s = [x \mapsto 0]$.

Solution of Exercise 3

1. Let $S = \text{while } x \geq y \text{ do } (x := x - 1; y := y + 1) \text{ and } s = [x \mapsto 3, y \mapsto 0]$. Then we have

$$\begin{aligned}
 \langle S, s \rangle &= \langle \text{while } x \geq y \text{ do } (x := x - 1; y := y + 1), [x \mapsto 3, y \mapsto 0] \rangle \\
 &\Rightarrow \langle \text{if } x \geq y \text{ then } x := x - 1; y := y + 1; S \text{ else skip}, [x \mapsto 3, y \mapsto 0] \rangle \quad (\text{WHILE}) \\
 &\Rightarrow \langle x := x - 1; y := y + 1; S, [x \mapsto 3, y \mapsto 0] \rangle \quad (\text{IF-TRUE}) \\
 &\Rightarrow \langle y := y + 1; S, [x \mapsto 2, y \mapsto 0] \rangle \quad (\text{COMP2 and ASS}) \\
 &\Rightarrow \langle \text{while } x \geq y \text{ do } (x := x - 1; y := y + 1), [x \mapsto 2, y \mapsto 1] \rangle \quad (\text{COMP-2 and ASS}) \\
 &\Rightarrow \langle \text{if } x \geq y \text{ then } x := x - 1; y := y + 1; S \text{ else skip}, [x \mapsto 2, y \mapsto 1] \rangle \quad (\text{WHILE}) \\
 &\Rightarrow \langle x := x - 1; y := y + 1; S, [x \mapsto 2, y \mapsto 1] \rangle \quad (\text{IF-TRUE}) \\
 &\Rightarrow \langle y := y + 1; S, [x \mapsto 1, y \mapsto 1] \rangle \quad (\text{COMP2 and ASS}) \\
 &\Rightarrow \langle \text{while } x \geq y \text{ do } (x := x - 1; y := y + 1), [x \mapsto 1, y \mapsto 2] \rangle \quad (\text{COMP-2 and ASS}) \\
 &\Rightarrow \langle \text{if } x \geq y \text{ then } x := x - 1; y := y + 1; S \text{ else skip}, [x \mapsto 1, y \mapsto 2] \rangle \quad (\text{WHILE}) \\
 &\Rightarrow \langle \text{skip}, [x \mapsto 1, y \mapsto 2] \rangle \quad (\text{IF-FALSE}) \\
 &\Rightarrow [x \mapsto 1, y \mapsto 2] \quad (\text{SKIP})
 \end{aligned}$$

2. $S = \text{if } (x \leq 3 \vee y > 2) \text{ then } (x := x + 3; y := 2) \text{ else skip}$ and $s = [x \mapsto 0]$. Then we have

$$\begin{aligned}
 \langle S, s \rangle &= \langle \text{if } (x \leq 3 \vee y > 2) \text{ then } (x := x + 3; y := 2) \text{ else skip}, [x \mapsto 0] \rangle \\
 &\Rightarrow \langle x := x + 3; y := 2, [x \mapsto 0] \rangle && \text{(IF-TRUE)} \\
 &\Rightarrow \langle y := 2, [x \mapsto 3] \rangle && \text{(COMP2 and ASS)} \\
 &\Rightarrow [x \mapsto 3, y \mapsto 2] && \text{(ASS)}
 \end{aligned}$$

Exercise 4

Consider the extension of **Bims** with the following new statement

$$S ::= \dots \mid \text{if } b \text{ then } S$$

which executes the statement S whenever the Boolean expression b is satisfied. Provide derivation rules that formalize the intuitive semantics explained above. Do it for both the big- and the small-step semantics cases.

Solution of Exercise 4

The rules to be added to the big-step semantics of **Bims** for the statement above are

$$\begin{array}{c}
 [\text{newIF-ttBS}] \frac{\langle S, s \rangle \rightarrow s' \quad s \vdash b \rightarrow_B tt}{\langle \text{if } b \text{ then } S, s \rangle \rightarrow s'} \qquad \frac{s \vdash b \rightarrow_B ff}{\langle \text{if } b \text{ then } S, s \rangle \rightarrow s} [\text{newIF-ffBS}]
 \end{array}$$

As for the small step semantics, we should add the following rules

$$\begin{array}{c}
 [\text{newIF-ttSS}] \frac{s \vdash b \rightarrow_B tt}{\langle \text{if } b \text{ then } S, s \rangle \Rightarrow \langle S, s \rangle} \qquad \frac{s \vdash b \rightarrow_B ff}{\langle \text{if } b \text{ then } S, s \rangle \Rightarrow s} [\text{newIF-ffSS}]
 \end{array}$$

Exercise 1*

Are the following statements well-typed? If so, prove it.

1. `var Int x := 2; var Int y := x + 1; skip`
2. `var Int x := 2; var Int x := 3; skip`
3. `var Bool x := false; while ¬x do skip`
4. `if x then x := ¬x else skip`
5. `var Int x := x + 1; skip`

Solution of Exercise 1

In the following, the rules (num) and (skip) for numeral integers and the skip statement will be applied without explicitly mentioning their use.

1. The derivation below proves that `var Int x := 2; var Int y := x + 1; skip` is well-typed (to fit the page limits, the derivation has been slit in two parts).

$$\begin{array}{c}
 \vdots \\
 \text{[dec]} \frac{x \notin \text{dom}(\emptyset) \quad \emptyset \vdash 2: \text{Int} \quad \overline{E \vdash \text{var Int } y := x + 1; \text{skip}: \text{ok}}}{\emptyset \vdash \text{var Int } x := 2; \text{var Int } y := x + 1; \text{skip}: \text{ok}} \\
 \\
 \text{[dec]} \frac{y \notin \text{dom}(E) \quad \begin{array}{c} \text{[var]} \frac{E(x) = \text{Int}}{E \vdash x: \text{Int}} \\ \text{[sum]} \frac{E \vdash x: \text{Int} \quad E \vdash 1: \text{Int}}{E \vdash x + 1: \text{Int}} \quad E' \vdash \text{skip}: \text{ok} \end{array}}{E \vdash \text{var Int } y := x + 1; \text{skip}: \text{ok}}
 \end{array}$$

where $E = [x \mapsto \text{Int}]$ and $E' = [x \mapsto \text{Int}, y \mapsto \text{Int}]$.

2. The statement `var Int x := 2; var Int x := 3; skip` is not well-typed as the variable x is defined twice and the type rule (dec) does not allow it. Below we highlight in **red** where the derivation fails:

$$\text{[dec]} \frac{x \notin \text{dom}(\emptyset) \quad \emptyset \vdash 2: \text{Int} \quad \overline{\text{red } x \notin \text{dom}(E) \quad \dots \quad E \vdash \text{var Int } x := 3; \text{skip}: \text{ok}} \text{[dec]}}{\emptyset \vdash \text{var Int } x := 2; \text{var Int } x := 3; \text{skip}: \text{ok}}$$

where $E = [x \mapsto \text{Int}]$.

3. The derivation proves that `var Bool x := false; while ¬x do skip` is well-typed.

$$\begin{array}{c}
 \text{[dec]} \frac{x \notin \text{dom}(\emptyset) \quad \emptyset \vdash \text{false: Bool} \quad \begin{array}{c} \text{[var]} \frac{E(x) = \text{Bool}}{E \vdash x: \text{Bool}} \\ \text{[neg]} \frac{E \vdash x: \text{Bool}}{E \vdash \neg x: \text{Bool}} \\ \text{[while]} \frac{E \vdash \neg x: \text{Bool} \quad E \vdash \text{skip: ok}}{E \vdash \text{while } \neg x \text{ do skip: ok}} \\ \hline \emptyset \vdash \text{var Bool } x := \text{false; while } \neg x \text{ do skip: ok}
 \end{array}
 \end{array}$$

where $E = [x \mapsto \text{Bool}]$.

Note that, the statement considered above always loops, nevertheless it is well-typed. This is because the type system we considered is not concerned with checking for this sort of behaviours.

4. The statement `if x then x := ¬x else skip` is not well-typed as the variable x has not be defined before its use. Below we highlight in **red** where the derivation fails:

$$\begin{array}{c}
 \text{[if]} \frac{\begin{array}{c} \text{[var]} \frac{\text{red } \emptyset(x) = \text{Bool}}{\emptyset \vdash x: \text{Bool}} \\ \text{[ass]} \frac{\text{red } \emptyset(x) = \text{Bool} \quad \begin{array}{c} \text{[var]} \frac{\text{red } \emptyset(x) = \text{Bool}}{\emptyset \vdash x: \text{Bool}} \\ \text{[neg]} \frac{\emptyset \vdash x: \text{Bool}}{E \vdash \neg x: \text{Bool}} \end{array}}{\emptyset \vdash x := \neg x: \text{ok}} \quad \emptyset \vdash \text{skip: ok}}{\emptyset \vdash \text{if } x \text{ then } x := \neg x \text{ else skip: ok}}
 \end{array}$$

5. The statement `var Int x := x + 1; skip` is not-well-typed as the variable x has not be defined before its use. Below we highlight in **red** where the derivation fails:

$$\begin{array}{c}
 \text{[dec]} \frac{x \notin \text{dom}(\emptyset) \quad \begin{array}{c} \text{[var]} \frac{\text{red } \emptyset(x) = \text{Int}}{\emptyset \vdash x: \text{Int}} \\ \text{[sum]} \frac{\emptyset \vdash x: \text{Int} \quad \emptyset \vdash 1: \text{ok}}{\emptyset \vdash x + 1: \text{Int}} \end{array} \quad [x \mapsto \text{Int}] \vdash \text{skip: ok}}{\emptyset \vdash \text{var Int } x := x + 1; \text{skip: ok}}
 \end{array}$$

Exercise 2

Let $S = \text{var Int } x := 1; \text{if } x < 1 \text{ then } x := \text{true} \text{ else } x := x + 1$. Prove that the statement S does not result in any execution error (i.e., show that it is not the case that $\emptyset \vdash \langle S, \emptyset \rangle \rightarrow !$) and that, at the same time, it is ill-typed.

Solution of Exercise 2

We first show that S is ill-typed. Let $\text{IF} = \text{if } x < 1 \text{ then } x := \text{true} \text{ else } x := x + 1$. Below we highlight in **red** where the derivation fails:

$$\frac{[dec] \quad \frac{x \notin \text{dom}(\emptyset) \quad \emptyset \vdash 1 : \text{Int}}{\emptyset \vdash \text{var Int } x := 1; \text{IF} : \text{ok}} \quad \frac{\dots \quad \frac{E(x) = \text{Int} \quad E \vdash \text{true} : \text{Bool}}{E \vdash x := \text{true} : \text{ok}} \quad [if]}{E \vdash \text{IF} : \text{ok}} \quad [ass]$$

where $E = [x \mapsto \text{Int}]$.

To show that it is not the case that $\emptyset \vdash \langle S, \emptyset \rangle \rightarrow!$ we proceed by contradiction and assume that there is a derivation that proves $\emptyset \vdash \langle S, \emptyset \rangle \rightarrow!$. Then, only one of the possible cases is possible:

Case 1 the last rule applied in the derivation is

$$\frac{\vdash 1 \rightarrow 1 \quad 1 \notin \text{set}(\text{Int})}{\emptyset \vdash \langle \text{var Int } x := 1; \text{IF}, \emptyset \rangle \rightarrow!}$$

This leads to a contradiction as $1 \in \text{set}(\text{Int}) = \mathbb{Z}$.

Case 2 the last rule applied in the derivation is

$$\frac{x \in \text{dom}(\emptyset)}{\emptyset \vdash \langle \text{var Int } x := 1; \text{IF}, \emptyset \rangle \rightarrow!}$$

This leads to a contradiction as $x \notin \text{dom}(\emptyset)$.

Case 3 the last rule applied in the derivation is

$$\frac{\emptyset \vdash 1 \rightarrow 1 \quad [x \mapsto \text{Int}] \vdash \langle \text{IF}, [x \mapsto 1] \rangle \rightarrow!}{\emptyset \vdash \langle \text{var Int } x := 1; \text{IF}, \emptyset \rangle \rightarrow!}$$

This leads us to three sub-cases, depending on which was the last rule that was applied to derive $[x \mapsto 1] \vdash \langle \text{IF}, \emptyset \rangle \rightarrow!$:

Sub-case 1: if the last rule was

$$\frac{\emptyset \vdash x \leq 1 \rightarrow \text{tt} \quad \text{tt} \notin \text{set}(\text{Bool})}{[x \mapsto \text{Int}] \vdash \langle \text{IF}, [x \mapsto 1] \rangle \rightarrow!}$$

This leads to a contradiction as $\text{tt} \in \text{set}(\text{Bool})$.

Sub-case 2: if the last rule was

$$\frac{[x \mapsto \text{Int}] \vdash \langle \mathbf{x} := \mathbf{true}, [x \mapsto 1] \rangle \rightarrow! \quad [x \mapsto 1] \vdash \mathbf{x} < 1 \rightarrow \mathbf{tt}}{[x \mapsto \text{Int}] \vdash \langle \mathbf{IF}, [x \mapsto 1] \rangle \rightarrow!}$$

This leads to a contradiction as $[x \mapsto 1] \vdash \mathbf{x} < 1 \not\rightarrow \mathbf{tt}$.

Sub-case 3: if the last rule was

$$\frac{\frac{[x \mapsto 1] \vdash \mathbf{x} \rightarrow 1 \quad [x \mapsto 1] \vdash 1 \rightarrow 1}{\emptyset \vdash \mathbf{x} + 1 \rightarrow 2} \quad 2 \notin \mathbf{safe}(\text{Int})}{\frac{[x \mapsto \text{Int}] \vdash \langle \mathbf{x} := \mathbf{x} + 1, [x \mapsto 1] \rangle \rightarrow! \quad [x \mapsto 1] \vdash \mathbf{x} < 1 \rightarrow \mathbf{ff}}{[x \mapsto \text{Int}] \vdash \langle \mathbf{IF}, [x \mapsto 1] \rangle \rightarrow!}}$$

This leads to a contradiction as $2 \in \mathbf{safe}(\text{Int})$.

Exercise 3

Define a type system for **Bims** (without variable declarations) that disallows statements with more than two nested while-loops. For example, `while true do (while true do skip)` is allowed, whereas `while true do (while true do (while true do skip))`.

(Hint: choose types and typing judgements that allow to count the number nestings).

Solution of Exercise 3

We define our new types $T \in \mathbf{Types}$ and typing judgements as follows

$$\begin{aligned} T &::= \bullet \mid \bullet\bullet \mid \bullet\bullet\bullet & (\text{types formation rules}) \\ \vdash S : T & & (\text{typing judgement}) \end{aligned}$$

We define an order on **Types** as follows: $\bullet \leq \bullet\bullet \leq \bullet\bullet\bullet$.

A type system solving the exercise is the following (other solutions are possible)

$$\begin{aligned} (\text{ass}) \quad & \frac{}{\vdash x := e : \bullet\bullet\bullet} & \frac{}{\vdash \text{skip} : \bullet\bullet\bullet} \quad (\text{skip}) \\ (\text{comp}) \quad & \frac{\vdash S_1 : T_1 \quad \vdash S_2 : T_2}{\vdash S_1; S_2 : \min(T_1, T_2)} & \frac{\vdash S_1 : T_1 \quad \vdash S_2 : T_2}{\vdash \text{if } e \text{ then } S_1 \text{ else } S_2 : \min(T_1, T_2)} \quad (\text{if}) \\ (\text{while-1}) \quad & \frac{\vdash S : \bullet\bullet\bullet}{\vdash \text{while } e \text{ do } S : \bullet\bullet} & \frac{\vdash S : \bullet\bullet}{\vdash \text{while } e \text{ do } S : \bullet} \quad (\text{while-2}) \end{aligned}$$

The number of dots are used to count the nestings of while-loops, where $\vdash S : \bullet$ means that the maximum number of nestings has been reached.

Exercise 1

Consider the **Bims** statements below

$$\begin{aligned} S_1 &= \mathbf{x} := 0; \text{ while } (\mathbf{x} \geq 0) \text{ do } \mathbf{x} := \mathbf{x} + 1 \\ S_2 &= \mathbf{x} := 3; \mathbf{y} := 4; \text{ while } (\mathbf{x} \leq \mathbf{y}) \text{ do } (\mathbf{x} := 2 * \mathbf{x}; \mathbf{y} := 3 * \mathbf{y}) \end{aligned}$$

and check whether they are semantically equivalent in the big-step semantics, that is $S_1 \sim_{bs} S_2$. Motivate your answer.

Solution of Exercise 1

By definition, $S_1 \sim_{bs} S_2$ if for all $s, s' \in \mathbf{States}$ we have

$$\langle S_1, s \rangle \rightarrow s' \text{ if and only if } \langle S_2, s \rangle \rightarrow s'.$$

We will prove that the above holds by showing that, for any $s, s' \in \mathbf{States}$, $\langle S_1, s \rangle \not\rightarrow s'$ and $\langle S_2, s \rangle \not\rightarrow s'$.

To show that $\langle S_2, s \rangle \not\rightarrow s'$ it suffices to prove that there is no finite proof tree for $\langle S_2, s \rangle \rightarrow s'$, for any $s, s' \in \mathbf{States}$. To this end, notice that for $s'' \in \mathbf{States}$ such that $0 \leq s''(x) \leq s''(y)$ we have that, for any $s' \in \mathbf{States}$,

$$\langle \text{while } (\mathbf{x} \leq \mathbf{y}) \text{ do } (\mathbf{x} := 2 * \mathbf{x}; \mathbf{y} := 3 * \mathbf{y}), s'' \rangle \not\rightarrow s',$$

since by $s''(x) \leq s''(y)$, only the [while-true] rule can be applied, and after the execution of the body of the while-loop we obtain the state $s''' = s''[x \mapsto 2s''(x), y \mapsto 3s''(y)]$ for which it holds again $0 \leq s'''(x) \leq s'''(y)$. Moreover, note that $\langle S_2, s \rangle \rightarrow s'$ requires us to show that

$$\frac{\langle \mathbf{x} := 3; \mathbf{y} := 4, s \rangle \rightarrow s'' \quad \langle \text{while } (\mathbf{x} \leq \mathbf{y}) \text{ do } (\mathbf{x} := 2 * \mathbf{x}; \mathbf{y} := 3 * \mathbf{y}), s'' \rangle \rightarrow s'}{\langle \mathbf{x} := 3; \mathbf{y} := 4; \text{ while } (\mathbf{x} \leq \mathbf{y}) \text{ do } (\mathbf{x} := 2 * \mathbf{x}; \mathbf{y} := 3 * \mathbf{y}), s \rangle \rightarrow s'}$$

where $s'' = s[x \mapsto 3, y \mapsto 4]$. By what we have said before, however, we have that the second premise of the rule cannot be proven, therefore

$$\langle \mathbf{x} := 3; \mathbf{y} := 4; \text{ while } (\mathbf{x} \leq \mathbf{y}) \text{ do } (\mathbf{x} := 2 * \mathbf{x}; \mathbf{y} := 3 * \mathbf{y}), s \rangle \not\rightarrow s'$$

Analogously, one can prove that $\langle S_1, s \rangle \not\rightarrow s'$, for all $s, s' \in \mathbf{States}$.

Exercise 2

Consider the **Bims** statements below, for (integer) numerals $n_1, n_2 \in \mathbf{Int}$

$$\begin{aligned} S_3 &= \mathbf{y} := n_1; \text{ for } \mathbf{x} := 1 \text{ to } n_2 \text{ do } \mathbf{y} := \mathbf{y} + 1 \\ S_4 &= \mathbf{y} := n_1 + n_2 * (n_2 + 1) / 2; \mathbf{x} := n_2 + 1. \end{aligned}$$

Check whether they are semantically equivalent in the big-step semantics, that is $S_3 \sim_{bs} S_4$, for all (integer) numerals $n_1, n_2 \in \mathbf{Int}$. Motivate your answer.

Solution of Exercise 2

S_3 and S_4 are not semantically equivalent. Take for instance $n_1 = 50$ and $n_2 = 2$. One can show that $\langle S_3, s \rangle \rightarrow s[x \mapsto 3, y \mapsto 52]$ whereas $\langle S_4, s \rangle \rightarrow s[x \mapsto 3, y \mapsto 53]$ (the proof trees for these derivations are left as exercise). This counterexample solves the exercise.

Exercise 3

Consider the following extension of **Bims** which adds the following formation rule to those of **Stm**, for $m > 0$,

$$S ::= \dots \mid \text{foreach } x \text{ in } [e_1, \dots, e_m] \text{ do } S.$$

Intuitively, the above construct executes the body S m -times, and at each execution of the body S , the value of the variable x is set to the value of the expression e_i , for $i = 1 \dots m$. At the end of the execution of the **foreach** construct, x assumes the value of the expression e_m . Give both the big-step and the small-step semantics that formalize the above description.

Solution of Exercise 3

The big-step semantics is given by the following two rules

$$\begin{aligned} [\text{FOREACH-1}_{BS}] \quad & \frac{\langle x := e_1; S; \text{foreach } x \text{ in } [e_2, \dots, e_m] \text{ do } S, s \rangle \rightarrow s'}{\langle \text{foreach } x \text{ in } [e_1, \dots, e_m] \text{ do } S, s \rangle \rightarrow s'} \\ [\text{FOREACH-2}_{BS}] \quad & \frac{\langle x := e; S, s \rangle \rightarrow s'}{\langle \text{foreach } x \text{ in } [e] \text{ do } S, s \rangle \rightarrow s'} \end{aligned}$$

The small-step semantics is given by the following two rules

$$\begin{aligned} [\text{FOREACH-1}_{SS}] \quad & \langle \text{foreach } x \text{ in } [e_1, \dots, e_m] \text{ do } S, s \rangle \Rightarrow \\ & \langle x := e_1; S; \text{foreach } x \text{ in } [e_2, \dots, e_m] \text{ do } S, s \rangle \\ [\text{FOREACH-2}_{SS}] \quad & \langle \text{foreach } x \text{ in } [e] \text{ do } S, s \rangle \Rightarrow \langle x := e; S, s \rangle \end{aligned}$$

Exercise 4

Consider the following statements in **Bims**

$$\begin{aligned} S_5 &= y := x + 4; (\text{for } x := 1 \text{ to } 3 \text{ do } y := y * x); y := y + x \\ S_6 &= (\text{if } x < 0 \text{ then } x := 2 * x \text{ else } x := 2 + x); x := x * (-1) \\ S_7 &= \text{while } (x < 200) \text{ do } S_6 \\ S_8 &= \text{repeat } S_6 \text{ until } (x \geq 200) \end{aligned}$$

Find all the transitions (if there are any) in the ss-semantics, for each of the following cases:

1. $\langle S_5, [x \mapsto -2] \rangle \Rightarrow^*?$
2. $\langle S_7, [x \mapsto 100] \rangle \Rightarrow^*?$
3. $\langle S_8, [x \mapsto 100] \rangle \Rightarrow^*?$

Solution of Exercise 4

$$\begin{aligned} \langle S_5, [x \mapsto -2] \rangle &\Rightarrow \langle (\text{for } x := 1 \text{ to } 3 \text{ do } y := y * x); y := y + x, [x \mapsto -2, y \mapsto 2] \rangle \\ &\Rightarrow \langle (\text{for } x := 2 \text{ to } 3 \text{ do } y := y * x); y := y + x, [x \mapsto 1, y \mapsto 2] \rangle \\ &\Rightarrow \langle (\text{for } x := 3 \text{ to } 3 \text{ do } y := y * x); y := y + x, [x \mapsto 2, y \mapsto 4] \rangle \\ &\Rightarrow \langle (\text{for } x := 4 \text{ to } 3 \text{ do } y := y * x); y := y + x, [x \mapsto 3, y \mapsto 12] \rangle \\ &\Rightarrow \langle y := y + x, [x \mapsto 4, y \mapsto 12] \rangle \\ &\Rightarrow [x \mapsto 4, y \mapsto 16] \end{aligned}$$

$$\begin{aligned} \langle S_7, [x \mapsto 100] \rangle &\Rightarrow \langle \text{if } x < 200 \text{ then } S_6; \text{ while } (x < 200) \text{ do } S_6 \text{ else skip}, [x \mapsto 100] \rangle \\ &\Rightarrow \langle S_6; \text{ while } (x < 200) \text{ do } S_6, [x \mapsto 100] \rangle \\ &\Rightarrow \langle x := 2 + x; x := x * (-1); \text{ while } (x < 200) \text{ do } S_6, [x \mapsto 100] \rangle \\ &\Rightarrow \langle x := x * (-1); \text{ while } (x < 200) \text{ do } S_6, [x \mapsto 102] \rangle \\ &\Rightarrow \langle \text{while } (x < 200) \text{ do } S_6, [x \mapsto -102] \rangle \\ &\Rightarrow \langle \text{if } x < 200 \text{ then } S_6; \text{ while } (x < 200) \text{ do } S_6 \text{ else skip}, [x \mapsto -102] \rangle \\ &\Rightarrow \langle S_6; \text{ while } (x < 200) \text{ do } S_6, [x \mapsto -102] \rangle \\ &\Rightarrow \langle x := 2 * x; x := x * (-1); \text{ while } (x < 200) \text{ do } S_6, [x \mapsto -102] \rangle \\ &\Rightarrow \langle x := x * (-1); \text{ while } (x < 200) \text{ do } S_6, [x \mapsto -204] \rangle \\ &\Rightarrow \langle \text{while } (x < 200) \text{ do } S_6, [x \mapsto 204] \rangle \\ &\Rightarrow \langle \text{if } x < 200 \text{ then } S_6; \text{ while } (x < 200) \text{ do } S_6 \text{ else skip}, [x \mapsto 204] \rangle \\ &\Rightarrow \langle \text{skip}, [x \mapsto 204] \rangle \\ &\Rightarrow [x \mapsto 204] \end{aligned}$$

First we define a SS-semantics for `repeat`:

$$[\text{REPEAT}_{SS}] \langle \text{repeat } S \text{ until } b, s \rangle \Rightarrow \langle S; \text{if } b \text{ then skip else (repeat } S \text{ until } b), s \rangle$$

$$\langle S_8, [x \mapsto 100] \rangle$$

$$\Rightarrow \langle S_6; \text{if } x \geq 200 \text{ then skip else (repeat } S_6 \text{ until } x \geq 200), [x \mapsto 100] \rangle$$

$$\Rightarrow \langle x := x + 2; x := x * (-1);$$

$$\quad \text{if } x \geq 200 \text{ then skip else (repeat } S_6 \text{ until } x \geq 200), [x \mapsto 100] \rangle$$

$$\Rightarrow \langle x := x * (-1); \text{if } x \geq 200 \text{ then skip else (repeat } S_6 \text{ until } x \geq 200), [x \mapsto 102] \rangle$$

$$\Rightarrow \langle \text{if } x \geq 200 \text{ then skip else (repeat } S_6 \text{ until } x \geq 200), [x \mapsto -102] \rangle$$

$$\Rightarrow \langle \text{repeat } S_6 \text{ until } x \geq 200, [x \mapsto -102] \rangle$$

$$\Rightarrow \langle S_6; \text{if } x \geq 200 \text{ then skip else (repeat } S_6 \text{ until } x \geq 200), [x \mapsto -102] \rangle$$

$$\Rightarrow \langle x := x + 2; x := x * (-1);$$

$$\quad \text{if } x \geq 200 \text{ then skip else (repeat } S_6 \text{ until } x \geq 200), [x \mapsto -102] \rangle$$

$$\Rightarrow \langle x := x * (-1); \text{if } x \geq 200 \text{ then skip else (repeat } S_6 \text{ until } x \geq 200), [x \mapsto -204] \rangle$$

$$\Rightarrow \langle \text{if } x \geq 200 \text{ then skip else (repeat } S_6 \text{ until } x \geq 200), [x \mapsto 204] \rangle$$

$$\Rightarrow \langle \text{skip}, [x \mapsto 204] \rangle$$

$$\Rightarrow [x \mapsto 204]$$

Exercise 1

Consider the following two (wrong!) variations of the (dec_{BS}) rule for variable declarations in the big-step semantics of **Bims** implementing the environment-store model (the changes to the original rule are highlighted in red):

$$\begin{aligned}
 [\text{dec}^*_{\text{BS}}] & \frac{\sigma \circ \mathcal{E} \vdash e \rightarrow_E v \quad \mathcal{E}[x \mapsto l] \vdash_l \langle S, \sigma[l \mapsto v] \rangle \rightarrow \sigma'}{\mathcal{E} \vdash_l \langle \text{var } T \ x := e; S, \sigma \rangle \rightarrow \sigma'} \\
 [\text{dec}^{**}_{\text{BS}}] & \frac{\sigma \circ \mathcal{E} \vdash e \rightarrow_E v \quad \mathcal{E} \vdash_{\text{nxt}(l)} \langle S, \sigma[l \mapsto v] \rangle \rightarrow \sigma'}{\mathcal{E} \vdash_l \langle \text{var } T \ x := e; S, \sigma \rangle \rightarrow \sigma'}
 \end{aligned}$$

What happens to the execution of the statement `var Int x := 2; var Int y := x + 1; skip` when the (dec_{BS}) is replaced by one of the rules above in the big-step semantics? Provide an intuitive general explanation and justify your claim with a derivation.

Solution of Exercise 1

Let $S = \text{var Int } x := 2; \text{var Int } y := x + 1; \text{skip}$.

Case 1. When $(\text{dec}^*_{\text{BS}})$ is used for (dec_{BS}) , all variables bindings are assigned to the same location. The effect is that, whenever a variable is reassigned, the reassignment is applied to all the variables declared up to the moment of the assignment. An example is provided by the following derivation:

$$[\text{dec}^*_{\text{BS}}] \frac{\sigma \circ \mathcal{E} \vdash 2 \rightarrow_E 2 \quad [\text{dec}^*_{\text{BS}}] \frac{\sigma_1 \circ \mathcal{E}_1 \vdash x + 1 \rightarrow_E 3 \quad \mathcal{E}_2 \vdash_l \langle \text{skip}, \sigma_2 \rangle \rightarrow \sigma_2}{\mathcal{E}_1 \vdash_l \langle \text{var Int } y := x + 1; \text{skip}, \sigma_1 \rangle \rightarrow \sigma_2}}{\mathcal{E} \vdash_l \langle S, \sigma \rangle \rightarrow \sigma_2}$$

where $\mathcal{E}_1 = \mathcal{E}[x \mapsto l]$, $\sigma_1 = \sigma[l \mapsto 2]$, $\mathcal{E}_2 = \mathcal{E}[x \mapsto l, y \mapsto l]$, and $\sigma_2 = \sigma[l \mapsto 3]$. Indeed, the variable bindings that we obtain at the computation (which are described by the variable environment $\mathcal{E}_2$) maps both x and y to the same location l , so the resulting state $\sigma_2 \circ \mathcal{E}_2$ assigns the same value to both variables.

Case 2. When $(\text{dec}^{**}_{\text{BS}})$ is used for (dec_{BS}) , the variable bindings don't change during the computation. This might result in not being able to evaluate expressions containing variables. An example is provided below, where we fail to conclude the derivation because the variable x cannot be evaluated when $x \notin \text{dom}(\mathcal{E})$:

$$[\text{dec}^*_{\text{BS}}] \frac{\sigma \circ \mathcal{E} \vdash 2 \rightarrow_E 2 \quad [\text{dec}^*_{\text{BS}}] \frac{[\text{var}] \frac{\sigma \circ \mathcal{E}(x) = ?}{\sigma \circ \mathcal{E} \vdash x \rightarrow_E ?} \quad [\text{sum}] \frac{\sigma \circ \mathcal{E} \vdash 1 \rightarrow_E 1}{\sigma_1 \circ \mathcal{E} \vdash x + 1 \rightarrow_E ?}}{\mathcal{E} \vdash_{l_2} \langle \text{skip}, \sigma' \rangle \rightarrow \sigma'} \quad \mathcal{E} \vdash_{l_1} \langle \text{var Int } y := x + 1; \text{skip}, \sigma_1 \rangle \rightarrow \sigma'}{\mathcal{E} \vdash_l \langle S, \sigma \rangle \rightarrow \sigma'}$$

where $l_i = \text{nxt}^i(l)$ (i -fold repetitions of the nxt function), $\sigma_1 = \sigma[l \mapsto 2]$ and σ' cannot be determined as we cannot evaluate the expression $x + 1$.

Exercise 2*

What is the expected evaluation of x at the end of the execution of the statement S below?

$S = \text{var Int } x := 2;$
 $\quad \text{begin } x := x + 1; \text{var Int } x := 2; \text{skip end};$
 $\quad x := x + 1$

Confirm your intuition by showing a derivation using the environment-store model semantics of **Bims** with variable scopes.

Solution of Exercise 2

The final evaluation of the global variable x is 4. We prove our claim by showing a derivation for $\mathcal{E} \vdash_l \langle S, \sigma \rangle \rightarrow \sigma'$, which corresponds to executing S under variable environment $\mathcal{E}$ (expressing the initial variable bindings) and store σ (expressing the initial configuration of the memory). To fit the page limit, we split the derivation in several parts.

Let $B = x := x + 1; \text{var Int } x := 2; \text{skip}$. Then

$$\begin{array}{c}
 \vdots \\
 \hline
 [\text{decBS}] \frac{\sigma \circ \mathcal{E} \vdash 2 \rightarrow_E 2 \quad \mathcal{E}_1 \vdash_{l_1} \langle \text{begin } B \text{ end}; x := x + 1, \sigma_1 \rangle \rightarrow \sigma'}{\mathcal{E} \vdash_l \langle S, \sigma \rangle \rightarrow \sigma'}
 \end{array}$$

$$\begin{array}{c}
 \vdots \\
 \hline
 [\text{blockBS}] \frac{\mathcal{E}_1 \vdash_{l_1} \langle B, \sigma_1 \rangle \rightarrow \sigma_3}{\mathcal{E}_1 \vdash_{l_1} \langle \text{begin } B \text{ end}, \sigma_1 \rangle \rightarrow \sigma_3} \quad [\text{assBS}] \frac{\sigma_3 \circ \mathcal{E}_1 \vdash x + 1 \rightarrow_E 4}{\mathcal{E}_1 \vdash_{l_1} \langle x := x + 1, \sigma_3 \rangle \rightarrow \sigma'} \\
 \hline
 [\text{compBS}] \frac{}{\mathcal{E}_1 \vdash_{l_1} \langle \text{begin } B \text{ end}; x := x + 1, \sigma_1 \rangle \rightarrow \sigma'}
 \end{array}$$

$$\begin{array}{c}
 [\text{assBS}] \frac{\sigma_1 \circ \mathcal{E}_1 \vdash x + 1 \rightarrow_E 3}{\mathcal{E}_1 \vdash_{l_1} \langle x := x + 1, \sigma_1 \rangle \rightarrow \sigma_2} \quad \frac{\sigma_2 \circ \mathcal{E}_1 \vdash 2 \rightarrow_E 2 \quad \mathcal{E}_2 \vdash_{l_2} \langle \text{skip}, \sigma_3 \rangle \rightarrow \sigma_3}{\mathcal{E}_1 \vdash_{l_1} \langle \text{var Int } x := 2; \text{skip}, \sigma_2 \rangle \rightarrow \sigma_3} [\text{decBS}] \\
 \hline
 [\text{compBS}] \frac{}{\mathcal{E}_1 \vdash_{l_1} \langle B, \sigma_1 \rangle \rightarrow \sigma_3}
 \end{array}$$

where $l_i = \text{nxt}^i(l)$ (i -fold repetitions of the nxt function) and

$$\begin{array}{ll}
 \mathcal{E}_1 = \mathcal{E}[x \mapsto l] & \sigma_1 = \sigma[l \rightarrow 2] \\
 \mathcal{E}_2 = \mathcal{E}[x \mapsto l_1] & \sigma_2 = \sigma[l \rightarrow 3] \\
 & \sigma_3 = \sigma[l \rightarrow 3, l_1 \mapsto 2] \\
 & \sigma' = \sigma[l \rightarrow 4, l_1 \mapsto 2]
 \end{array}$$

As the variable bindings that are in place at the end of the execution are described by $\mathcal{E}_1$, we have that the value of x is $\sigma' \circ \mathcal{E}_1(x) = 4$, as claimed.

Exercise 3*

Consider the statement defined below

```
var Int  $x := \underline{0}$ ;  
var Int  $y := \underline{1}$ ;  
begin  
  proc  $p$  is  $x := x + \underline{1}$ ;  
  var Int  $x := \underline{3}$ ;  
  call  $p$   
end;  
 $y := x$ 
```

What is the final value assigned of the (global) variable y under the following scope rules?

1. fully static scope rules
2. mixed scope rules: dynamic variable bindings and static procedure name bindings
3. fully dynamic scope rules

Solution of Exercise 3

1. The final value of y is 1. Under fully static scope rules, calling p corresponds to update the value of the global variable of x (which is the variable binding at the time of declaration).
2. The final value of y is 0. Under dynamic variable bindings, as the local declaration of x inside the block is shadowing the global one, calling p corresponds to update the local variable x . This update, though, has no effects once we exit the block, as y is assigned the value of the global variable x .
3. The final value of y is 0. As p does not contain calls of other procedures, the effect is the same as the one explained in (2).

Exercise 4

What is the execution of `proc  $p$  is call  $p$ ; call  $p$` in the big-step semantics of **Bip** under fully static scope rules?

We show that $S = \text{proc } p \text{ is call } p; \text{ call } p$ loops forever in the big-step semantics. Assume, for the sake of finding a contradiction that $\mathcal{E}, \pi \vdash_l \langle S, \sigma \rangle \rightarrow \sigma'$ can be derived. Then the only possible derivation tree for it looks like the following

$$\frac{\vdots}{\frac{[\text{call}_{\text{BS}}] \frac{\mathcal{E}, \pi' \vdash_l \langle \text{call } p, \sigma \rangle \rightarrow \sigma' \quad \pi'(p) = (\text{call } p, \mathcal{E}, \pi)}{[\text{proc}_{\text{BS}}] \frac{\mathcal{E}, \pi' \vdash_l \langle \text{call } p, \sigma \rangle \rightarrow \sigma'}{\mathcal{E}, \pi \vdash_l \langle S, \sigma \rangle \rightarrow \sigma'}}$$

However, due to its circularity, this would be an infinite derivation. Thus no finite derivation for $\mathcal{E}, \pi \vdash_l \langle S, \sigma \rangle \rightarrow \sigma'$ exist. From this we conclude that S loops forever.

Let S be the statement from Exercise 3. Construct a derivation for $\mathcal{E}, \pi \vdash_l \langle S, \sigma \rangle \rightarrow \sigma'$ (if it exists) in the big-step semantics when fully static scope rules are chosen.

Let $B = (\text{proc } p \text{ is } x := x + \underline{1}; \text{var Int } x := \underline{3}; \text{call } p)$ be the body of the block environment. The derivation for $\mathcal{E}, \pi \vdash_l \langle S, \sigma \rangle \rightarrow \sigma'$ is shown below (zoom-in for details):

$$\begin{array}{c}
\frac{\sigma_3 \circ \mathcal{E}_2 \vdash x + \underline{1} \rightarrow_E 1}{[\text{ass}_{\text{BS}}]} \\
\frac{\mathcal{E}_2, \pi_1 \vdash_{l_3} \langle x := x + \underline{1}, \sigma_3 \rangle \rightarrow \sigma_4 \quad \pi_1(p) = (x := x + \underline{1}, \mathcal{E}_2, \pi)}{[\text{call}_{\text{BS}}]} \\
\frac{\sigma_2 \circ \mathcal{E}_2 \vdash \underline{3} \rightarrow_E 3 \quad \mathcal{E}_3, \pi_1 \vdash_{l_3} \langle \text{call } p, \sigma_3 \rangle \rightarrow \sigma_4}{[\text{dec}_{\text{BS}}]} \\
\frac{\mathcal{E}_2, \pi_1 \vdash_{l_2} \langle \text{var Int } x := \underline{3}; \text{call } p, \sigma_2 \rangle \rightarrow \sigma_4}{[\text{proc}_{\text{BS}}]} \\
\frac{\mathcal{E}_2, \pi \vdash_{l_2} \langle B, \sigma_2 \rangle \rightarrow \sigma_4}{[\text{block}_{\text{BS}}]} \\
\frac{\mathcal{E}_2, \pi \vdash_{l_2} \langle \text{begin } B \text{ end}, \sigma_2 \rangle \rightarrow \sigma_4}{[\text{comp}_{\text{BS}}]} \\
\frac{\sigma_4 \circ \mathcal{E}_2 \vdash x + \underline{1} \rightarrow_E 4 \quad \mathcal{E}_2, \pi \vdash_{l_2} \langle y := x, \sigma_4 \rangle \rightarrow \sigma'}{[\text{ass}_{\text{BS}}]} \\
\frac{\sigma_1 \circ \mathcal{E}_1 \vdash \underline{1} \rightarrow_E 1 \quad \mathcal{E}_2, \pi \vdash_{l_2} \langle \text{begin } B \text{ end}; y := x, \sigma_2 \rangle \rightarrow \sigma'}{[\text{dec}_{\text{BS}}]} \\
\frac{\sigma \circ \mathcal{E} \vdash \underline{0} \rightarrow_E 0 \quad \mathcal{E}_1, \pi \vdash_{l_1} \langle \text{var Int } y := \underline{1}; \text{begin } B \text{ end}; y := x, \sigma_1 \rangle \rightarrow \sigma'}{[\text{dec}_{\text{BS}}]} \\
\mathcal{E}, \pi \vdash_{l_1} \langle \text{var Int } x := 0; \text{var Int } y := \underline{1}; \text{begin } B \text{ end}; y := x, \sigma \rangle \rightarrow \sigma'
\end{array}$$

$$\mathcal{E}_1 = \mathcal{E}[x \mapsto l]$$

$$\mathcal{E}_2 = \mathcal{E}[x \mapsto l, y \mapsto l_1] \quad \pi_1 = \pi[p \mapsto (x := x + \underline{1}, \mathcal{E}_2, \pi)]$$

$$\mathcal{E}_3 = \mathcal{E}[x \mapsto l_2, y \mapsto l_1]$$

$$\sigma_1 = \sigma[l \mapsto 0]$$

$$\sigma_2 = \sigma[l \mapsto 0, l_1 \mapsto 1]$$

$$\sigma_3 = \sigma[l \mapsto 0, l_1 \mapsto 1, l_2 \mapsto 3]$$

$$\sigma_4 = \sigma[l \mapsto 1, l_1 \mapsto 1, l_2 \mapsto 3]$$

$$\sigma' = \sigma[l \mapsto 2, l_1 \mapsto 1, l_2 \mapsto 3]$$

Exercise 1*

Consider the statement below

```

var Int x := 2;
var Int y := 5;
proc p(strgy Int x) is x := x + 1;
proc q(strgy Int y) is call p(y); y := x + 1;
call q(x)

```

What are the final values assigned to the (global) variables x and y under the following different parameter-passing strategies? For each case, justify your answer by showing the resulting derivation. (Assume fully-static scope rule).

1. call-by-reference (i.e., replace **strgy** with **ref**);
2. call-by-value (i.e., replace **strgy** with **val**);
3. call-by-name (i.e., replace **strgy** with **name**).

Solution of Exercise 1

We show only the part that differs in the derivations, that is, from the point where the procedure q is invoked. This corresponds to provide a derivation for

$$\mathcal{E}, \pi \vdash_{l_3} \langle \text{call } q(x), \sigma \rangle \rightarrow \sigma',$$

where $\mathcal{E} = [x \mapsto l_1, y \mapsto l_2]$, $\sigma = [l_1 \mapsto 2, l_2 \mapsto 5]$, $\pi' = [p \mapsto (x := x + 1, x, \mathcal{E}, \emptyset)]$, and $\pi = \pi'[q \mapsto (\text{call } p(y) \ y := x + 1, y, \mathcal{E}, \pi')]$. (As usual, $l_{i+1} = \text{nxt}(l_i)$).

1. Call-by-reference

$$\begin{array}{c}
\frac{\sigma \circ \mathcal{E}_2 \vdash x + 1 \rightarrow_E 3}{\text{[assBS]} \quad \frac{\mathcal{E}_2, \pi' \vdash_{l_3} \langle x := x + 1, \sigma \rangle \rightarrow \sigma_1}{\text{[call-REFBS]} \quad \frac{\mathcal{E}_1, \pi \vdash_{l_3} \langle \text{call } p(y), \sigma \rangle \rightarrow \sigma_1}{\text{[compBS]} \quad \frac{\mathcal{E}_1, \pi \vdash_{l_3} \langle \text{call } p(y); y := x + 1, \sigma \rangle \rightarrow \sigma_2}{\text{[call-REFBS]} \quad \frac{\mathcal{E}_1, \pi \vdash_{l_3} \langle \text{call } p(y); y := x + 1, \sigma \rangle \rightarrow \sigma_2}{\mathcal{E}, \pi \vdash_{l_3} \langle \text{call } q(x), \sigma \rangle \rightarrow \sigma_2}}}}
\end{array}$$

where

$$\begin{array}{ll}
\mathcal{E}_1 = \mathcal{E}[y \mapsto \mathcal{E}(x)] = [x \mapsto l_1, y \mapsto l_1] & \sigma_1 = [l_1 \mapsto 3, l_2 \mapsto 5] \\
\mathcal{E}_2 = \mathcal{E}[x \mapsto \mathcal{E}_1(y)] = [x \mapsto l_1, y \mapsto l_2] & \sigma_2 = [l_1 \mapsto 4, l_2 \mapsto 5] \\
\mathcal{E}_3 = [x \mapsto l_1, y \mapsto l_1] &
\end{array}$$

Thus, at the end of the call, x is assigned the value 4 and y keeps value 5.

2. Call-by-value (we split the derivation in 3 parts)

$$\begin{array}{c}
\vdots \\
\hline
[\text{call-VAL}_{\text{BS}}] \frac{\mathcal{E}_1, \pi \vdash_{l_4} \langle \text{call } p(y); y := x + 1, \sigma_1 \rangle \rightarrow \sigma_4 \quad \sigma \circ \mathcal{E} \vdash x \rightarrow_E 2}{\mathcal{E}, \pi \vdash_{l_3} \langle \text{call } q(x), \sigma \rangle \rightarrow \sigma_4} \\
\\
\begin{array}{c}
\vdots \\
\hline
[\text{comp}_{\text{BS}}] \frac{\mathcal{E}_1, \pi \vdash_{l_4} \langle \text{call } p(y), \sigma_1 \rangle \rightarrow \sigma_3 \quad [\text{ass}_{\text{BS}}] \frac{\sigma_3 \circ \mathcal{E}_1 \vdash x + 1 \rightarrow_E 3}{\mathcal{E}_1, \pi \vdash_{l_4} \langle y := x + 1, \sigma_3 \rangle \rightarrow \sigma_4}}{\mathcal{E}_1, \pi \vdash_{l_4} \langle \text{call } p(y); y := x + 1, \sigma_1 \rangle \rightarrow \sigma_4}
\end{array} \\
\\
[\text{call-VAL}_{\text{BS}}] \frac{[\text{ass}_{\text{BS}}] \frac{\sigma_2 \circ \mathcal{E}_2 \vdash x + 1 \rightarrow_E 3}{\mathcal{E}_2, \pi' \vdash_{l_5} \langle x := x + 1, \sigma_2 \rangle \rightarrow \sigma_3} \quad \sigma_1 \circ \mathcal{E}_1 \vdash y \rightarrow_E 2}{\mathcal{E}_1, \pi \vdash_{l_4} \langle \text{call } p(y), \sigma_1 \rangle \rightarrow \sigma_3}
\end{array}$$

where

$$\begin{array}{ll}
\mathcal{E}_1 = \mathcal{E}[y \mapsto l_3] = [x \mapsto l_1, y \mapsto l_3] & \sigma_1 = [l_1 \mapsto 2, l_2 \mapsto 5, l_3 \mapsto 2] \\
\mathcal{E}_2 = \mathcal{E}[x \mapsto l_4] = [x \mapsto l_4, y \mapsto l_2] & \sigma_2 = [l_1 \mapsto 2, l_2 \mapsto 5, l_3 \mapsto 2, l_4 \mapsto 2] \\
& \sigma_3 = [l_1 \mapsto 2, l_2 \mapsto 5, l_3 \mapsto 2, l_4 \mapsto 3] \\
& \sigma_4 = [l_1 \mapsto 2, l_2 \mapsto 5, l_3 \mapsto 3, l_4 \mapsto 2]
\end{array}$$

Thus, at the end of the call, the values of x and y are left unchanged.

3. Call-by-name

$$\begin{array}{c}
[\text{ass}_{\text{BS}}] \frac{\sigma \circ \mathcal{E} \vdash x + 1 \rightarrow_E 3}{\mathcal{E}, \pi' \vdash_{l_3} \langle x := x + 1, \sigma \rangle \rightarrow \sigma_1} \\
[\text{call-NAME}_{\text{BS}}] \frac{\mathcal{E}_1, \pi \vdash_{l_3} \langle \text{call } p(x), \sigma \rangle \rightarrow \sigma_1 \quad [\text{ass}_{\text{BS}}] \frac{\sigma_1 \circ \mathcal{E} \vdash x + 1 \rightarrow_E 4}{\mathcal{E}_1, \pi \vdash_{l_3} \langle y := x + 1, \sigma_1 \rangle \rightarrow \sigma_2}}{\mathcal{E}_1, \pi \vdash_{l_3} \langle \text{call } p(x); y := x + 1, \sigma \rangle \rightarrow \sigma_2} \\
[\text{comp}_{\text{BS}}] \frac{\mathcal{E}_1, \pi \vdash_{l_3} \langle \text{call } p(x); y := x + 1, \sigma \rangle \rightarrow \sigma_2}{\mathcal{E}, \pi \vdash_{l_3} \langle \text{call } q(x), \sigma \rangle \rightarrow \sigma_2}
\end{array}$$

where $\sigma_1 = [l_1 \mapsto 3, l_2 \mapsto 5]$, $\sigma_2 = [l_1 \mapsto 3, l_2 \mapsto 4]$ and we observe that

$$\begin{aligned}
(\text{call } p(y); y := x + 1)[y \mapsto x] &= \text{call } p(x); y := x + 1 \\
(x := x + 1)[x \mapsto x] &= x := x + 1.
\end{aligned}$$

Thus, at the end of the call, x gets assigned value 3 and y value 4.

Exercise 2

What is the result of applying the following capture-avoiding substitutions?

1. $(x := 3)[x \mapsto y]$;
2. $(\text{proc } p(\text{val Int } x) \text{ is } x := y + 1)[y \mapsto z]$;
3. $(\text{proc } p(\text{val Int } x) \text{ is } x := y + 1)[y \mapsto x + 1]$;
4. $(\text{begin } x := x + y; \text{var Int } x := x + y; \text{skip end})[x \mapsto y]$.

Solution of Exercise 2

1. $x := 3$ (the left-hand-side of an assignment is not subject to substitution);
2. $\text{proc } p(\text{val Int } x) \text{ is } x := z + 1$ (the variable y is free, so can be safely substituted);
3. $\text{proc } p(\text{val Int } z) \text{ is } z := (x + 1) + 1$ (we first apply α -renaming, replacing a fresh variable z for x , then apply the substitution. Observe that this prevents x to be captured by the name binding of the procedure declaration);
4. $\text{begin } x := y + y; \text{var Int } x := x + y; \text{skip end}$ (in the first assignment x is free; while in the second assignment x is bound by a new declaration).

Exercise 3

Call-by-macro-expansion is a parameter passing strategy similar to call by name, but uses syntactic substitution rather than capture-avoiding substitution. Propose big-step semantics for it and provide an instance of a program that executes differently if call-by-name strategy is chosen instead of call-by-macro-expansion.

Solution of Exercise 3

We assume the syntax for procedure declaration is $\text{proc } p(\text{macroT } x) \text{ is } S_1; S_2$ (we add a “tag” to clarify the parameter passing strategy we want to use) and show only the rule for fully-static scope rule (the other cases are similar). The only change we need to address in the semantics proposed in class is rule for the call statement (the format of transitions is the same):

$$[\text{call-ME}_{\text{BS}}] \frac{\mathcal{E}', \pi[p \mapsto (S, x, \mathcal{E}', \pi')] \vdash_l \langle S[e/x], \sigma \rangle \rightarrow \sigma' \quad \pi(p) = (S, x, \mathcal{E}', \pi')}{\mathcal{E}, \pi \vdash_l \langle \text{call } p(e), \sigma \rangle \rightarrow \sigma'}$$

where $S[e/x]$ denotes the syntactic substitution of x for e in all expressions occurring in the statement S .

A simple program showing the differences between call-by-macro-expansion and call-by-name is:

```
var Int y := 5;  
proc p(strgy Int x) is var Int x := 1; y := x  
call p(y + 1)
```

where $\text{strgy} \in \{\text{macro}, \text{name}\}$. Indeed, at the end of the call of p , for the case of call-by-name the value of y is 1; and for call-by-macro-expansion is 6.